

GROWTHBOT DE

Engineering Handbook v1.0

Consolidated internal handbook for mission, architecture, product engineering, AI platform, security, testing, observability, backend, performance, deployment, governance, and future platform evolution.

VERSION

1.0

STATUS

Structural baseline frozen

BRANCH POLICY

develop -> main

Table of Contents

1. Engineering Handbook

2. Project Mission

3. Architecture

4. Product Engineering

5. AI Platform

6. Backend API

7. Security

8. Performance

9. Testing

10. Frontend

11. Observability

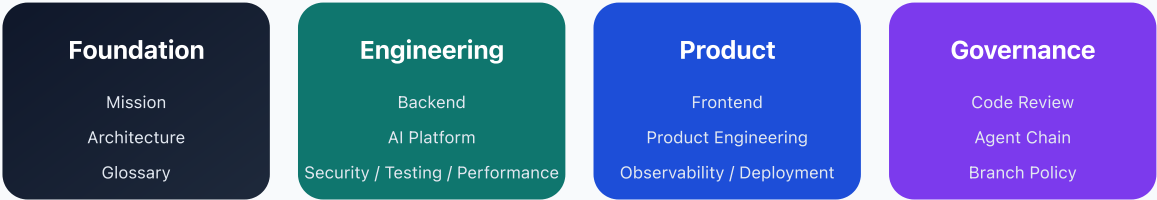
12. Deployment

13. Code Review

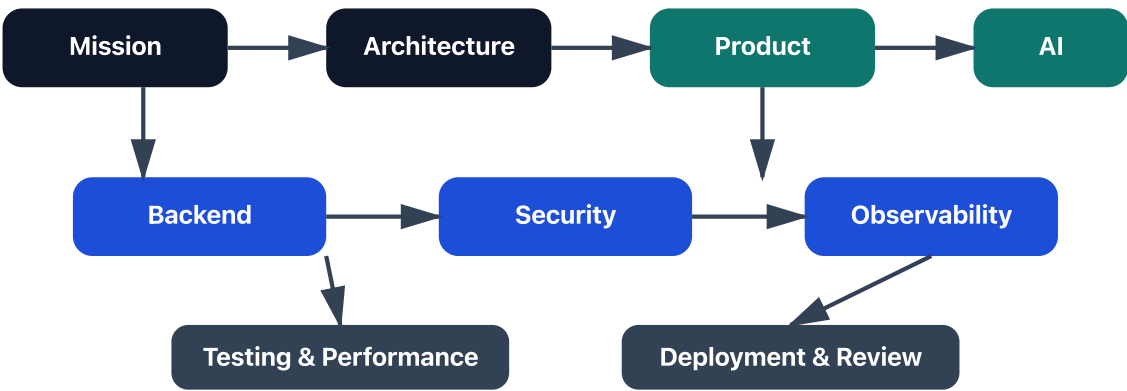
14. Engineering Glossary

15. Agent Chain

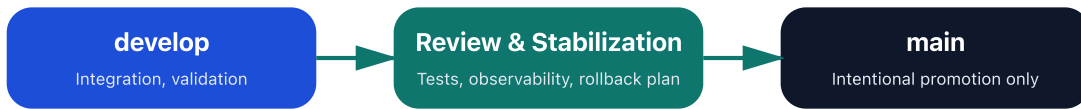
Engineering Handbook Pillars



Conceptual Reading Order



Official Promotion Flow



README.MD

Engineering Handbook

Engineering Handbook

Este diretório contém todas as regras oficiais de engenharia do GrowthBot DE.

Nenhum agente de IA ou desenvolvedor deve implementar funcionalidades sem considerar estes documentos.

A estrutura está organizada em quatro pilares:

- Foundation
- Engineering
- Product
- Governance

Cada documento possui uma responsabilidade específica e complementa os demais.

O objetivo deste handbook é garantir consistência arquitetural, qualidade técnica, escalabilidade e alinhamento entre produto, engenharia e Inteligência Artificial.

Política de branches do projeto:

- `develop` é a branch principal de evolução contínua
- toda mudança deve ser integrada primeiro em `develop`
- `main` deve receber apenas mudanças promovidas a partir de `develop`
- alterações diretas em `main` devem ser evitadas e só podem acontecer em situações excepcionais e controladas

Project Mission

Project Mission

Este documento define a identidade, a missão e a direção estratégica do GrowthBot DE. Ele é a referência principal para alinhar produto, engenharia, IA e evolução de longo prazo da plataforma.

Objetivo

Definir com clareza o que o GrowthBot DE é, por que ele existe, qual valor ele pretende gerar e quais princípios devem orientar sua evolução como produto.

Este documento existe para impedir deriva estratégica. Sempre que houver dúvida entre uma decisão tecnicamente possível e uma decisão coerente com a visão do produto, a missão definida aqui deve prevalecer.

Escopo

Este documento cobre:

- posicionamento do produto
- visão central da plataforma
- missão do GrowthBot DE
- filosofia comercial
- definição de valor
- direção de evolução do produto
- critérios estratégicos para orientar decisões futuras

Este documento não define regras detalhadas de arquitetura, implementação, segurança, testes ou deploy. Esses temas pertencem aos demais documentos do Engineering Handbook.

Regras

1. Posicionamento do Produto

GrowthBot DE é uma AI Sales Platform.

Seu propósito não é apenas responder perguntas.

Seu propósito é entender intenção do cliente, qualificar oportunidades, recomendar a melhor solução, orientar decisões de compra e maximizar resultados comerciais mensuráveis por meio do uso responsável de Inteligência Artificial.

O projeto não deve ser tratado conceitualmente como:

- um chatbot simples
- um assistente genérico
- um recomendador isolado
- uma integração pontual com LLM

Esses elementos podem existir como partes do sistema, mas não definem sua identidade.

O produto deve ser sempre pensado como plataforma.

2. Natureza da Plataforma

GrowthBot DE deve evoluir como um sistema comercial orientado por intenção, decisão e mensuração.

Isso significa que a plataforma precisa, ao longo do tempo, ser capaz de:

- compreender intenção conversacional
- transformar intenção em contexto acionável

- selecionar recomendações de forma responsável
- guiar a jornada do usuário
- registrar sinais relevantes do comportamento
- permitir acompanhamento operacional
- alimentar automações e otimizações futuras

A IA é parte central da plataforma, mas não é o produto inteiro.

O valor real do GrowthBot DE está na combinação entre inteligência, estrutura, rastreabilidade, confiança e operação.

3. Missão

A missão do GrowthBot DE é transformar intenção em valor.

Esse valor deve ser gerado simultaneamente para:

- usuário
- operador
- empresa
- parceiros

Esse valor deve surgir por meio de:

- decisões inteligentes
- recomendações personalizadas
- automação responsável
- experiência clara e confiável
- visibilidade operacional sobre o funil e seus resultados

Receita é importante, mas não é a definição completa de sucesso.

Receita deve ser consequência de um sistema que gera valor real e confiança ao longo da jornada.

4. Filosofia Comercial

O objetivo do GrowthBot DE nunca será vender qualquer produto.

O objetivo do sistema será recomendar a melhor solução possível para cada cliente dentro do contexto disponível.

Uma venda só deve ser considerada verdadeiramente bem-sucedida quando existir alinhamento entre:

- necessidade do cliente
- qualidade da recomendação
- confiança da IA
- experiência do usuário
- interesse comercial

A receita deve ser consequência da confiança construída durante a conversa e reforçada ao longo da jornada.

Isso significa que o sistema não deve ser otimizado apenas para empurrar ofertas.

Ele deve ser otimizado para relevância, aderência, qualidade de recomendação e coerência comercial.

5. Uso Responsável de IA

A Inteligência Artificial no GrowthBot DE deve ser usada como mecanismo de apoio à decisão, personalização e eficiência, nunca como desculpa para comportamento opaco, enganoso ou irresponsável.

O uso de IA deve sempre respeitar:

- clareza de propósito
- segurança operacional
- coerência com a necessidade do usuário
- possibilidade de evolução controlada
- responsabilidade sobre os efeitos gerados

Nenhuma decisão de produto deve assumir que “usar IA” por si só gera valor.

O valor surge quando a IA melhora entendimento, recomendação, experiência e resultado mensurável sem comprometer confiança.

6. Critério Central de Valor

O GrowthBot DE deve existir para reduzir distância entre intenção e decisão útil.

Isso significa que a plataforma precisa ajudar o usuário a sair de um estado de dúvida, interesse difuso ou necessidade mal formulada para um estado de maior clareza, confiança e direcionamento.

Sempre que houver escolha entre:

- mais volume e menos qualidade
- mais agressividade e menos confiança
- mais automação e menos controle
- mais conversão aparente e menos valor real

o produto deve favorecer qualidade, confiança, clareza e sustentabilidade.

7. Princípio de Evolução do Produto

O GrowthBot DE não deve ser pensado apenas como o que é hoje.

Sua evolução natural pode seguir a seguinte trajetória:

- Chatbot
- AI Assistant
- AI Sales Assistant
- AI Sales Agent
- AI Sales Platform
- AI Revenue Platform

Essa progressão não significa que todas as etapas serão implementadas imediatamente.

Ela significa que a arquitetura, o produto e a tomada de decisão devem evitar escolhas que bloqueiem essa evolução futura.

8. Visão Futura

Na sua visão mais madura, o GrowthBot DE deve ser capaz de gerenciar o ciclo comercial de forma muito mais ampla do que simples recomendação.

Isso inclui a capacidade de:

- entender intenção do cliente
- qualificar oportunidades
- recomendar a melhor solução
- lidar com objeções dentro de limites definidos
- acompanhar o cliente com follow-ups
- medir conversões, custos e eficiência
- otimizar continuamente o funil com base em métricas e sinais observáveis

Essa visão final se aproxima do conceito de AI Revenue Platform.

Nesse estágio, o sistema não apenas conversa. Ele opera, aprende, mede e melhora o fluxo comercial de forma estruturada.

9. Relação Entre Produto e Engenharia

Toda decisão de engenharia deve servir ao produto.

Toda decisão de produto deve respeitar a realidade de operação, rastreabilidade, manutenção e evolução tecnológica.

O GrowthBot DE não deve crescer por acúmulo de features desconectadas. Ele deve crescer como sistema coerente.

Se uma nova ideia:

- não melhora entendimento do usuário

- não melhora qualidade da recomendação
- não melhora confiança
- não melhora visibilidade operacional
- não melhora a capacidade de aprender com o funil

então essa ideia deve ser reavaliada antes de entrar no roadmap.

10. Critério de Coerência Estratégica

Toda mudança importante no projeto deve responder a pelo menos uma destas perguntas:

- isso melhora a qualidade da recomendação?
- isso melhora a experiência do usuário?
- isso melhora a capacidade de medir o sistema?
- isso melhora a operação da plataforma?
- isso fortalece a evolução de longo prazo do produto?

Se a resposta for não, a mudança pode estar introduzindo esforço sem reforçar a identidade real do GrowthBot DE.

Project Mission Decision Framework

Antes de introduzir mudança estratégica relevante, deve ser possível responder:

1. isso reforça o GrowthBot DE como AI Sales Platform?
2. isso transforma intenção em valor de forma mais clara?
3. isso melhora usuário, operador, empresa ou parceiros?
4. isso fortalece confiança, clareza ou mensuração?
5. existe risco de reduzir o produto a chatbot ou integração isolada?
6. essa decisão preserva a visão de longo prazo?

Project Mission Anti-Patterns

Nunca:

- tratar o GrowthBot DE como chatbot simples
- otimizar apenas para conversão imediata
- usar IA sem propósito claro
- introduzir mudanças que enfraquecem confiança
- crescer por acúmulo de features desconectadas

Future Compatibility

A missão do produto deve permanecer compatível com a evolução futura da plataforma para contextos mais amplos de operação comercial, automação, mensuração e coordenação orientada por IA.

Definition of Done

Uma decisão estratégica só pode ser considerada coerente com a missão quando:

- reforça a identidade do produto
- aumenta valor real ou capacidade futura clara
- preserva confiança
- não contradiz a visão de longo prazo
- evita anti-patterns estratégicos

Checklist

- A decisão proposta reforça o GrowthBot DE como AI Sales Platform?
- A mudança ajuda a transformar intenção em valor?
- O impacto no usuário foi considerado além da conversão imediata?

- A recomendação continua orientada por aderência e confiança, e não apenas por venda?
- A decisão preserva a visão de longo prazo da plataforma?
- A proposta melhora clareza, confiança, mensuração ou capacidade operacional?
- A mudança evita reduzir o produto a um simples chatbot ou integração com LLM?
- O uso de IA nesta decisão é justificável, responsável e útil?

Observações

Este documento deve ser lido como referência estratégica central do Engineering Handbook.

Arquitetura, produto, IA, observabilidade e operação devem evoluir em coerência com esta missão.

Quando houver conflito entre ganho tático de curto prazo e coerência estratégica de longo prazo, a coerência estratégica deve prevalecer.

Architecture

Architecture

Este documento define as regras arquiteturais do GrowthBot DE. Ele é a principal referência para mudanças estruturais e para a evolução segura da plataforma ao longo do tempo.

Objetivo

Definir os princípios, limites e critérios arquiteturais que orientam a evolução do GrowthBot DE como AI Sales Platform. Este documento existe para garantir consistência estrutural, reduzir acoplamento indevido e permitir crescimento contínuo sem reescritas completas.

Escopo

Este documento cobre arquitetura de software, organização modular, regras de dependência, critérios para abstrações, evolução estrutural, desenho de serviços, relação com persistência, uso de eventos e requisitos mínimos de observabilidade arquitetural.

Ele não define missão do produto, regras de negócio detalhadas, padrões visuais de frontend ou instruções específicas de implementação. Esses tópicos pertencem a outros documentos da base [.ai-rules/](#).

Regras

1. Direção Arquitetural

GrowthBot DE deve ser tratado arquiteturalmente como uma AI Sales Platform, e não como um bot isolado, um assistente simples ou uma integração pontual com LLM.

Toda decisão arquitetural deve assumir que o sistema evoluirá para suportar:

- múltiplos canais de entrada
- múltiplos LLMs
- múltiplos MCP Servers
- múltiplos tenants
- banco compartilhado
- serviços distribuídos
- filas
- workers
- analytics
- automações

A arquitetura deve permitir crescimento incremental. Mudanças futuras devem ser acomodadas por extensão estrutural, e não por reescrita ampla do núcleo do sistema.

2. Princípios Arquiteturais

2.1 Modularidade

O sistema deve ser dividido em módulos com fronteiras claras. Cada módulo existe para resolver um problema delimitado e deve poder evoluir com o mínimo possível de impacto colateral.

Modularidade não significa multiplicar arquivos ou camadas sem necessidade. Significa estabelecer unidades de responsabilidade compreensíveis, previsíveis e sustentáveis.

2.2 Alta Coesão

Cada módulo deve concentrar elementos que pertencem ao mesmo contexto funcional. Arquivos, regras e fluxos relacionados a um mesmo domínio devem permanecer próximos.

Um módulo coeso facilita manutenção, leitura e testes porque o comportamento relevante está concentrado onde faz sentido.

2.3 Baixo Acoplamento

Módulos não devem depender de detalhes internos uns dos outros. Eles devem interagir por contratos simples e previsíveis.

Baixo acoplamento reduz impacto de mudanças locais, facilita substituição de implementações e melhora a capacidade de evolução incremental.

2.4 Single Responsibility

Cada unidade de software deve ter uma razão principal para mudar. Um arquivo, serviço ou módulo não deve acumular responsabilidades de naturezas diferentes apenas por conveniência.

Se uma alteração em regras de negócio, persistência, integração externa e interface afeta o mesmo arquivo, a responsabilidade provavelmente está mal definida.

2.5 Separation of Concerns

Interface, orquestração, regras de negócio, integrações externas, persistência e observabilidade devem permanecer separadas.

Separar preocupações não é burocracia. É o mecanismo que impede que o sistema se torne difícil de entender e perigoso de alterar.

2.6 Composition over Inheritance

A composição deve ser preferida à herança como forma de construir comportamento. Dependências explícitas e composição de serviços tendem a ser mais previsíveis, mais testáveis e menos rígidas.

Herança só deve aparecer quando houver uma justificativa muito clara e um ganho concreto de modelagem.

2.7 Evolução Incremental

A arquitetura deve ser preparada para crescer por pequenas decisões consistentes. Em vez de antecipar toda complexidade futura, o sistema deve abrir caminhos para evolução controlada.

O objetivo não é prever tudo. O objetivo é não bloquear o que provavelmente virá.

2.8 Simplicidade Antes da Complexidade

A solução mais simples que respeita os limites arquiteturais deve ser preferida. Complexidade só é aceitável quando reduz risco real, resolve restrição concreta ou prepara um caminho inevitável de evolução.

Sofisticação estrutural sem necessidade comprovada é dívida cognitiva.

2.9 Arquitetura Orientada ao Domínio

A organização deve refletir o domínio do negócio antes de refletir tecnologia. O eixo principal do projeto deve ser o fluxo comercial e operacional da plataforma, e não o framework utilizado.

Quando houver dúvida entre estruturar por camada técnica ou por contexto de negócio, a preferência deve ser por domínio, desde que a clareza seja mantida.

2.10 Dependências Apontando para o Núcleo do Negócio

O núcleo da lógica de negócio deve ser o elemento mais estável do sistema. Integrações, frameworks e detalhes de infraestrutura devem apontar para ele, e não o contrário.

A regra central é simples: tecnologia externa pode conhecer o domínio; o domínio não deve conhecer detalhes da tecnologia externa.

2.11 Interfaces Somente Quando Houver Necessidade Real

Interfaces, contratos e portas de abstração devem ser introduzidos quando houver motivo concreto, como múltiplas implementações, redução clara de acoplamento ou proteção de uma fronteira importante.

Não criar interfaces por hábito, por estilo ou por expectativa abstrata de futura necessidade.

2.12 Abstrações Apenas Quando Justificadas

Toda abstração deve pagar seu custo cognitivo. Uma abstração sem justificativa tende a esconder fluxo, aumentar indireção e tornar mudanças mais lentas.

Abstrair cedo demais é uma forma comum de overengineering.

3. Organização dos Módulos

Cada módulo deve possuir responsabilidade única e fronteira explícita.

Exemplos de domínios esperados para a evolução do projeto:

- Conversation
- Intent
- Recommendations
- Catalog
- AI Platform
- Tracking
- Follow-up
- Analytics
- Persistence
- Authentication
- Dashboard

Cada domínio deve evoluir como unidade relativamente independente. Isso significa:

- não misturar regras de múltiplos domínios no mesmo módulo
- não usar um módulo como “depósito genérico” de lógica variada
- não centralizar decisões de negócio de todos os contextos em um único arquivo ou serviço

Se uma implementação começa a atender mais de um domínio sem justificativa clara, ela deve ser reavaliada e potencialmente reorganizada.

4. Regras de Dependência

As dependências devem seguir um fluxo previsível.

A lógica de negócio não pode depender diretamente de:

- framework web
- banco de dados
- Telegram
- OpenAI
- MCP
- SDKs externos

Essas integrações devem permanecer isoladas em adapters, providers, gateways ou camadas equivalentes.

O núcleo do negócio deve operar sobre intenções, recomendações, eventos, sessões, decisões e estados do sistema, e não sobre detalhes específicos de transporte, SDK ou persistência.

Qualquer dependência externa que entre diretamente no núcleo tende a dificultar testes, migração tecnológica e evolução estrutural.

5. Responsabilidade dos Componentes

Cada arquivo deve possuir uma responsabilidade clara.

Devem ser evitados arquivos que simultaneamente:

- buscam dados
- transformam dados
- executam regras de negócio
- renderizam interface
- chamam APIs

Essa mistura de responsabilidades reduz previsibilidade, dificulta testes, espalha efeitos colaterais e torna manutenção mais cara.

Componentes, serviços, repositories, providers e módulos devem existir com fronteiras entendíveis e consistentes com sua função real.

6. Abstrações

Não criar abstrações apenas por elegância arquitetural.

Uma abstração só deve nascer quando existir pelo menos um dos seguintes fatores:

- duplicação real
- mais de uma implementação
- necessidade futura comprovada
- redução clara de acoplamento

Evitar overengineering é uma regra arquitetural explícita.

Abstrações mal justificadas:

- escondem fluxo sem necessidade
- criam múltiplos pontos de leitura para entender algo simples
- dificultam onboarding
- aumentam o custo de alteração

7. Regras de Evolução

Antes de criar um novo módulo, serviço ou camada, responder:

- Já existe um módulo responsável?
- Estou duplicando responsabilidade?
- Posso reutilizar uma implementação existente?
- Estou aumentando complexidade?
- Essa mudança melhora manutenção?

Se a resposta indicar duplicação, sobreposição de escopo ou complexidade sem ganho claro, a mudança deve ser repensada.

Arquitetura não deve crescer por impulso. Deve crescer por necessidade estruturada.

8. Regras para Novos Serviços

Todo novo serviço deve possuir definição explícita de:

- objetivo
- entradas
- saídas
- dependências
- falhas possíveis
- métricas
- testes

Serviços que nascem sem esse enquadramento tendem a virar pontos opacos do sistema.

O serviço não precisa necessariamente começar complexo, mas deve nascer inteligível. A clareza da responsabilidade é obrigatória desde o primeiro dia.

9. Regras para Banco de Dados

A lógica do domínio não pode conhecer detalhes do banco.

Toda persistência deve passar por repositories ou por uma camada equivalente que concentre acesso a dados e proteja o restante da aplicação de detalhes operacionais.

Regras obrigatórias:

- não espalhar SQL pela aplicação
- não misturar queries com regras de negócio

- não acoplar fluxo de domínio a particularidades de um banco específico
- projetar sempre pensando em futura migração de banco

Persistência é infraestrutura. O domínio consome capacidades de armazenamento; ele não deve depender dos detalhes concretos de como isso é feito.

10. Eventos

A arquitetura deve estimular uso de eventos como forma de tornar o sistema mais observável, rastreável e pronto para automações futuras.

Eventos importantes esperados no ecossistema do projeto:

- SessionStarted
- IntentDetected
- RecommendationGenerated
- RecommendationAccepted
- RecommendationRejected
- AffiliateClicked
- FollowupScheduled
- FollowupSent
- ConversionRegistered

Toda alteração relevante no fluxo comercial ou operacional deve ser elegível para emissão de evento observável.

Eventos não devem ser tratados como detalhe periférico. Eles são parte da arquitetura da plataforma porque conectam operação, analytics, automação e evolução futura.

11. Observabilidade Arquitetural

Todo fluxo importante deve ser rastreável.

Observabilidade deve existir para aumentar clareza operacional, debugar comportamento, medir qualidade do funil e sustentar evolução da plataforma.

Diretrizes obrigatórias:

- registrar apenas informações úteis
- nunca registrar segredos
- nunca registrar tokens
- nunca registrar prompts completos
- nunca registrar dados sensíveis sem necessidade e controle explícito
- criar correlação entre eventos usando IDs consistentes

Observabilidade não é ruído. Também não é logging indiscriminado. Ela deve ser estruturada para sustentar compreensão real do sistema.

12. Critério Arquitetural de Qualidade

Uma mudança estrutural só deve ser considerada boa quando melhora pelo menos alguns dos seguintes eixos sem degradar os demais de forma desproporcional:

- clareza
- manutenção
- separação de responsabilidades
- testabilidade
- capacidade de evolução
- previsibilidade
- observabilidade

Mudanças que apenas “parecem mais sofisticadas” não são melhorias arquiteturais por si só.

Architecture Decision Framework

Antes de introduzir mudança estrutural relevante, deve ser possível responder:

1. qual boundary está sendo afetado?
2. a responsabilidade está no módulo correto?
3. a mudança reduz ou aumenta acoplamento?
4. existe abstração desnecessária?
5. a evolução futura fica mais fácil ou mais difícil?
6. a arquitetura ficou mais clara ou apenas mais sofisticada?

Architecture Anti-Patterns

Nunca:

- misturar domínio com framework
- espalhar persistência pela aplicação
- criar abstrações sem necessidade real
- usar módulo genérico como depósito de lógica
- sacrificar clareza por sofisticação aparente

Future Compatibility

A arquitetura deve permanecer compatível com crescimento incremental da plataforma, múltiplos canais, múltiplos modelos, múltiplos runtimes, serviços mais distribuídos e maior observabilidade operacional.

Definition of Done

Uma mudança arquitetural só pode ser considerada concluída quando:

- respeita boundaries claros
- reduz ou controla acoplamento
- melhora clareza e manutenção
- preserva testabilidade e observabilidade
- evita anti-patterns arquiteturais

Checklist

- A mudança respeita fronteiras de domínio?
- O núcleo de negócio continua isolado de frameworks e SDKs externos?
- Existe responsabilidade clara para cada módulo afetado?
- A solução evita acoplamento desnecessário?
- Houve criação de abstração sem justificativa real?
- A persistência continua protegida por repositories?
- O fluxo alterado continua observável?
- Há eventos relevantes para mudanças importantes de estado?
- A mudança prepara evolução futura sem adicionar complexidade excessiva?
- A arquitetura ficou mais clara, e não apenas mais sofisticada?

Observações

Este documento deve ser tratado como referência arquitetural principal do projeto.

Nenhuma alteração estrutural relevante deve ignorar estas regras.

Quando houver conflito entre conveniência local e integridade arquitetural, a integridade arquitetural deve prevalecer.

Product Engineering

Product Engineering

Este documento define como produto, engenharia, experiência, operação e objetivos comerciais devem se conectar no GrowthBot DE. Ele estabelece a filosofia de Product Engineering da plataforma e orienta como funcionalidades devem ser pensadas, priorizadas, implementadas e evoluídas.

Objetivo

Definir as regras que garantem que toda decisão técnica produza valor real para o produto.

O papel deste documento é impedir duas distorções comuns:

- produto pensado sem rigor estrutural
- engenharia evoluindo sem clareza de valor

No GrowthBot DE, Product Engineering representa a união entre visão de produto, responsabilidade comercial, qualidade da experiência, sustentabilidade operacional e disciplina técnica.

Escopo

Este documento cobre:

- filosofia de produto do GrowthBot DE
- critérios de valor
- relação entre engenharia e produto
- princípios de priorização
- regras para evolução de funcionalidades
- qualidade de recomendação
- experiência do usuário
- conversão com responsabilidade
- automação orientada a valor
- relação entre confiança, receita e operação

Este documento não substitui a missão do projeto, não redefine arquitetura de software e não detalha a implementação da AI Platform. Ele atua entre missão e execução, conectando intenção de produto com decisões práticas de engenharia.

Regras

1. Product Engineering é parte central do projeto

No GrowthBot DE, Product Engineering não é etapa posterior à implementação.

É a disciplina que garante que a implementação faça sentido.

Toda funcionalidade, automação, integração ou fluxo de IA deve responder a uma pergunta simples:

isso melhora o produto de forma real, mensurável e sustentável?

Se a resposta não estiver clara, a iniciativa ainda não está madura.

2. O GrowthBot DE não é um chatbot

GrowthBot DE não deve ser pensado como chatbot simples, assistente genérico ou recomendador isolado.

Ele é uma AI Sales Platform.

Sua função é compreender intenção humana, organizar contexto relevante, recomendar a solução mais adequada, apoiar a jornada comercial e transformar sinais conversacionais em valor operacional.

Isso significa que o produto não termina na resposta.

O produto inclui:

- interpretação
- recomendação
- rastreamento
- follow-up
- visibilidade operacional
- capacidade de melhoria contínua

3. O objetivo central do produto é transformar intenção em valor

O objetivo do GrowthBot DE não é apenas converter.

O objetivo é transformar intenção em valor para:

- usuário
- operador
- empresa
- parceiros

Valor, neste contexto, significa entregar utilidade, clareza, confiança, eficiência operacional e resultado comercial responsável.

Receita é consequência desse processo, não sua justificativa única.

4. Toda funcionalidade deve melhorar pelo menos um pilar do produto

Nenhuma funcionalidade deve entrar apenas porque parece interessante, moderna ou tecnicamente elegante.

Toda alteração relevante deve fortalecer pelo menos um dos pilares abaixo:

- experiência do usuário
- qualidade da recomendação
- conversão
- retenção ou re-engajamento
- receita
- eficiência operacional
- visibilidade do sistema
- escalabilidade do produto
- confiabilidade da plataforma

Se uma mudança não melhora nenhum desses pilares, sua prioridade deve ser questionada.

5. Confiança vale mais que conversão cega

O GrowthBot DE não deve maximizar conversão sacrificando qualidade da recomendação.

Uma recomendação só deve ser considerada bem-sucedida quando existir alinhamento entre:

- necessidade real do usuário
- adequação da solução sugerida
- confiança suficiente na resposta
- integridade da experiência
- interesse comercial legítimo

Ganhos imediatos obtidos por recomendação inadequada degradam o produto no médio prazo.

6. O produto deve orientar, não empurrar

O GrowthBot DE não existe para vender qualquer produto para qualquer pessoa.

Sua função é ajudar o usuário a chegar à melhor decisão possível dentro das opções disponíveis.

Isso exige que o produto:

- respeite contexto
- reduza ruído
- aumente clareza
- preserve coerência
- evite manipulação comercial indevida

A venda deve ser consequência de uma boa orientação.

7. A qualidade da recomendação é ativo estrutural

No contexto do GrowthBot DE, qualidade da recomendação não é detalhe de copy.

Ela é núcleo do produto.

Uma recomendação de qualidade depende da combinação entre:

- boa leitura de intenção
- contexto suficiente
- catálogo coerente
- critérios claros de seleção
- mensagem adequada ao estágio da conversa
- integridade comercial

Se a qualidade da recomendação cair, o produto inteiro se degrada.

8. Experiência do usuário é parte da engenharia do produto

Experiência do usuário não deve ser tratada como camada decorativa ou etapa final.

Ela se expressa em:

- clareza da conversa
- previsibilidade do fluxo
- entendimento do próximo passo
- qualidade da recomendação
- redução de fricção
- tempo para valor
- consistência da interface operacional

Toda decisão técnica que prejudica compreensão, clareza ou continuidade também prejudica o produto.

9. Product Engineering exige equilíbrio entre curto e longo prazo

O produto precisa evoluir com velocidade suficiente para aprender, mas sem comprometer sua base estrutural.

Por isso, toda priorização deve equilibrar:

- ganho de curto prazo
- dívida estrutural criada
- risco operacional
- capacidade futura de expansão
- custo de manutenção

Nem toda entrega rápida é boa entrega.

Nem toda arquitetura elegante gera valor no momento certo.

10. Automação deve servir ao produto, não substituí-lo

Automações são importantes no GrowthBot DE, especialmente em follow-up, tracking, classificação e operação.

Mas automação só tem valor quando melhora:

- eficiência
- escala

- consistência
- rastreabilidade
- experiência

Automatizar processos ruins apenas acelera erros.

11. O produto deve ser pensado como sistema de jornada

Cada funcionalidade deve ser avaliada dentro de uma jornada maior.

A plataforma não deve ser otimizada apenas para o momento da resposta.

Ela deve considerar etapas como:

- entrada do usuário
- interpretação inicial
- recomendação
- clique
- re-engajamento
- decisão posterior
- análise operacional

Uma funcionalidade localmente boa pode ser ruim se quebrar a jornada completa.

12. Product Engineering precisa de mensuração

Sem medição, o produto vira opinião.

Toda funcionalidade relevante deve ser pensada com possibilidade futura de observar:

- adoção
- taxa de uso
- impacto em recomendação
- impacto em clique
- impacto em follow-up
- impacto em conversão
- impacto em tempo operacional
- impacto em qualidade percebida

Se não é possível medir nada sobre uma funcionalidade, sua utilidade real fica estruturalmente fraca.

13. Engenharia deve reduzir fricção para evolução do produto

A arquitetura e a implementação devem facilitar:

- testar hipóteses
- ajustar regras
- trocar critérios de recomendação
- inserir novos canais
- melhorar prompts
- revisar automações
- expandir visibilidade operacional

Quando uma simples mudança de produto exige alterações frágeis em muitas partes do sistema, a engenharia deixou de servir ao produto.

14. Nem toda possibilidade deve virar funcionalidade

O fato de algo ser tecnicamente possível não significa que deva ser implementado.

O GrowthBot DE deve evitar:

- features sem hipótese clara

- automações sem valor comprovável
- interface sem necessidade real
- IA sem função definida
- complexidade antecipada sem sinal de demanda

Disciplina de produto significa saber o que não construir.

15. O produto deve preservar coerência operacional

Toda funcionalidade introduz também custo operacional.

Por isso, além de valor percebido pelo usuário, Product Engineering deve considerar:

- capacidade de suporte
- facilidade de entendimento pelo operador
- visibilidade do estado do sistema
- impacto em manutenção
- impacto em análise futura

Uma funcionalidade comercialmente interessante, mas operacionalmente opaca, pode piorar o produto.

16. Product Engineering vem antes da AI Platform no raciocínio

Conceitualmente, o produto deve definir o que precisa ser resolvido antes da IA definir como isso será apoiado.

Isso significa:

- primeiro entender o objetivo do fluxo
- depois definir o valor esperado
- depois definir a experiência desejada
- depois definir a operação necessária
- só então decidir como a IA participa

A IA deve servir ao produto.

O produto não deve se deformar para caber na IA.

17. Cada entrega deve preservar opcionalidade futura

O GrowthBot DE deve crescer sem bloquear futuras evoluções relevantes, como:

- novos canais
- novos modelos
- novos catálogos
- novas jornadas
- novos mecanismos de tracking
- novas automações
- novos painéis operacionais

Isso não exige construir tudo agora.

Exige não fechar portas importantes por decisões curtas demais.

18. Definition of Value antes de Definition of Done

Antes de declarar uma entrega como pronta, deve estar claro qual valor ela pretendia gerar.

No mínimo, cada mudança relevante deve responder:

1. qual problema de produto ela resolve?
2. para quem ela gera valor?
3. como esse valor aparecerá na prática?
4. qual risco ela adiciona?
5. como saberemos se ela funcionou?

Sem definição de valor, Definition of Done vira apenas checklist técnico.

19. O produto deve evoluir por clareza, não por impulso

O GrowthBot DE deve crescer com intencionalidade.

A pergunta central nunca deve ser:

o que mais podemos adicionar?

A pergunta correta deve ser:

o que mais precisamos estruturar para aumentar valor, confiança e resultado de forma sustentável?

Product Decision Framework

Antes de criar qualquer funcionalidade relevante, a decisão deve passar por um framework explícito de produto.

No mínimo, deve ser possível responder:

1. existe problema real?
2. existe hipótese clara?
3. existe usuário real para isso?
4. existe KPI relacionado?
5. existe forma de medir resultado?
6. existe custo justificável?
7. existe impacto operacional compreensível?
8. existe risco relevante?
9. existe alternativa mais simples?
10. vale a pena construir isso agora?

Esse framework existe para impedir desenvolvimento impulsivo, orientado apenas por possibilidade técnica, preferência pessoal ou entusiasmo momentâneo.

Nem toda boa ideia merece implementação imediata.

Product Anti-Patterns

O Product Engineering do GrowthBot DE deve evitar explicitamente padrões que degradam clareza, foco e sustentabilidade do produto.

Nunca:

- criar funcionalidades sem hipótese
- duplicar funcionalidades
- criar telas sem uso real
- introduzir IA sem necessidade clara
- otimizar apenas para comissão
- introduzir configurações desnecessárias
- aumentar complexidade sem benefício proporcional
- confundir quantidade de funcionalidades com evolução do produto
- lançar fluxos sem critério de valor
- tratar backlog como prova de maturidade do produto

Esses anti-patterns devem ser tratados como sinais de alerta estrutural.

Quando aparecem com frequência, o problema geralmente não está na execução pontual, mas na ausência de disciplina de produto.

Future Compatibility

O Product Engineering deve preservar compatibilidade conceitual com a visão futura da plataforma.

Isso inclui preparação para cenários como:

- multi-tenant
- multi-language
- multi-channel
- novos parceiros
- marketplace
- novos modelos comerciais
- SaaS
- afiliados
- APIs públicas
- produtos digitais
- produtos físicos

Preparação para compatibilidade futura não significa implementar essas frentes agora.

Significa não bloquear estruturalmente essas possibilidades caso elas passem a fazer sentido para a evolução real do GrowthBot DE.

Definition of Done

Uma alteração relevante no produto só pode ser considerada concluída quando, de forma proporcional ao seu escopo:

- resolve um problema real
- está alinhada à missão do projeto
- respeita a arquitetura
- melhora pelo menos um pilar do produto
- preserva confiança do usuário
- possui impacto operacional compreensível
- possui medição possível ou planejada
- possui documentação suficiente
- possui testes compatíveis com o risco
- não introduz complexidade injustificável

Checklist

- A mudança resolve um problema real de produto?
- Existe hipótese clara para justificar essa mudança?
- Existe usuário real ou contexto real para essa funcionalidade?
- Está claro para quem essa mudança gera valor?
- A funcionalidade melhora pelo menos um pilar central do produto?
- A recomendação ou experiência ficam melhores com essa mudança?
- Existe risco de conversão cega ou degradação de confiança?
- O impacto operacional é compreensível?
- Existe KPI ou forma concreta de medir resultado?
- Existe alternativa mais simples que deveria ser preferida?
- Existe possibilidade de medir o resultado dessa mudança?
- A IA está servindo ao produto, e não conduzindo o produto?
- A complexidade criada é proporcional ao valor esperado?
- A mudança evita os anti-patterns de Product Engineering?
- A entrega preserva compatibilidade futura relevante?
- A entrega atende ao Definition of Done de Product Engineering?

Observações

Conceitualmente, este documento deve ser lido antes do documento de AI Platform.

Primeiro o GrowthBot DE define como o produto pensa.

Depois a plataforma de IA define como pode apoiar esse produto com inteligência, contexto, ferramentas e automação.

Sempre que houver tensão entre uma decisão tecnicamente interessante e uma decisão melhor para o produto, a segunda deve prevalecer.

AI Platform

AI Platform

Este documento define as regras da plataforma de IA do GrowthBot DE. Ele orienta como modelos, prompts, contexto, ferramentas, memória, MCP Servers e agentes devem ser estruturados para gerar recomendações confiáveis, auditáveis e evolutivas.

Objetivo

Definir os princípios e regras que governam o uso de Inteligência Artificial dentro do GrowthBot DE como AI Sales Platform.

Este documento existe para impedir que a camada de IA evolua de forma improvisada, opaca ou excessivamente acoplada ao restante do sistema. O objetivo não é apenas integrar modelos, mas estruturar uma plataforma de decisão assistida por IA que seja segura, mensurável, extensível e operacionalmente responsável.

Escopo

Este documento cobre:

- papel da IA dentro da plataforma
- organização da camada de modelos
- uso de prompts
- gestão de contexto
- regras para memória
- uso de ferramentas
- uso de MCP Servers
- desenho de agentes
- critérios para inferência
- tolerância a falhas da camada de IA
- requisitos de rastreabilidade e governança

Este documento não define missão do produto, estrutura geral de frontend, contratos detalhados de API, políticas completas de segurança ou processos de deploy. Esses tópicos pertencem aos demais documentos do Engineering Handbook.

Regras

1. Papel da IA na plataforma

A IA é componente central do GrowthBot DE, mas não é o produto inteiro.

Seu papel é ampliar a capacidade do sistema de:

- interpretar intenção
- organizar contexto
- apoiar recomendações
- melhorar qualidade de resposta
- permitir automação responsável
- aumentar eficiência operacional

A IA não deve ser tratada como camada mágica, autônoma ou infalível. Ela faz parte de um sistema maior, composto também por regras de negócio, dados, rastreamento, persistência, observabilidade e validação operacional.

Toda decisão de IA deve ser pensada como parte de um fluxo comercial mensurável.

2. A camada de IA deve ser modular

A plataforma de IA deve ser organizada em responsabilidades separadas.

No mínimo, o sistema deve distinguir conceitualmente:

- seleção e chamada de modelo
- construção de prompt
- montagem de contexto
- uso de ferramentas
- interpretação de saída
- validação de resposta
- persistência de sinais relevantes

Não misturar, em um mesmo ponto do sistema, regras de negócio, montagem de prompt, chamada de modelo, parsing de resposta e efeitos colaterais operacionais sem separação explícita.

Quanto mais a plataforma crescer, mais importante será isolar essas responsabilidades.

3. Modelos devem ser tratados como dependências substituíveis

Nenhum fluxo crítico deve nascer acoplado permanentemente a um único modelo, provedor ou formato específico de resposta.

A arquitetura da AI Platform deve assumir, desde cedo, que o projeto poderá evoluir para:

- múltiplos modelos
- múltiplos provedores
- múltiplas estratégias de inferência
- múltiplos custos e níveis de latência
- múltiplas políticas de fallback

Isso não significa criar abstrações excessivas antes da hora.

Significa evitar dependências estruturais irreversíveis.

Trocar de modelo, ajustar temperatura, mudar formato de saída ou introduzir fallback entre provedores deve ser uma evolução controlada, e não uma reescrita do sistema.

4. IA deve operar sobre contexto estruturado

Modelos não devem receber contexto bruto sem critério.

Todo contexto enviado para inferência deve ser selecionado, reduzido e organizado com intenção clara.

O sistema deve distinguir entre:

- contexto atual da interação
- histórico conversacional relevante
- dados persistidos do usuário
- catálogo ou base de produtos
- sinais operacionais
- regras de negócio aplicáveis
- instruções permanentes do sistema

Mais contexto nem sempre significa melhor inferência.

A qualidade da AI Platform depende da capacidade de enviar o contexto certo, no momento certo, na forma certa.

5. Prompt engineering é disciplina de produto e engenharia

Prompts não devem ser tratados como texto improvisado embutido em fluxos críticos sem critério de manutenção.

Prompts representam parte do comportamento do sistema e devem ser tratados como artefatos de engenharia.

Todo prompt relevante deve:

- ter objetivo claro
- operar dentro de um contexto delimitado
- refletir regras do produto

- minimizar ambiguidade
- restringir saídas inválidas quando necessário
- ser fácil de revisar e evoluir

Prompts longos, confusos, redundantes ou contraditórios aumentam custo, instabilidade e dificuldade de manutenção.

O GrowthBot DE deve preferir prompts explícitos, estruturados e orientados a comportamento desejado.

6. Saídas do modelo não devem ser aceitas cegamente

Toda saída de IA que influencie recomendação, automação, tracking, follow-up, qualificação ou decisão operacional deve passar por validação adequada.

Isso inclui, quando aplicável:

- validação de formato
- validação de campos obrigatórios
- validação semântica mínima
- checagem contra entidades existentes
- fallback seguro em caso de inconsistência

O modelo pode sugerir, mas o sistema é responsável por decidir se a resposta pode ou não ser promovida para uso operacional.

IA sem validação é risco estrutural.

7. Ferramentas devem ampliar capacidade, não mascarar fragilidade

Ferramentas existem para ampliar a capacidade do sistema de consultar, agir ou enriquecer respostas com base em informação externa ou operacional.

Ferramentas não devem ser introduzidas para compensar falta de arquitetura, falta de dados organizados ou ausência de contratos claros.

Antes de adicionar uma tool, deve estar claro:

- qual problema ela resolve
- qual dado ela consome
- qual efeito ela produz
- qual limite operacional ela possui
- como sua execução será rastreada
- como falhas serão tratadas

Ferramentas que executam ações externas devem operar com limites explícitos.

8. MCP Servers são parte formal da AI Platform

MCP Servers devem ser tratados como componentes oficiais de expansão de contexto e capacidade da plataforma de IA.

Seu uso deve seguir os mesmos critérios de rigor aplicados a qualquer integração estrutural:

- propósito claro
- autenticação segura
- escopo mínimo de acesso
- previsibilidade de uso
- rastreabilidade
- manutenção controlada

Nem todo MCP disponível deve ser integrado.

O critério não é quantidade de ferramentas. O critério é valor real com risco controlado.

9. Princípio do menor privilégio para ferramentas e MCPs

Toda ferramenta, MCP Server ou integração acionável deve receber apenas o nível mínimo de permissão necessário para cumprir sua função.

Isso vale para:

- leitura de arquivos
- acesso a repositórios
- acesso a documentação
- acesso a navegadores
- acesso a sistemas externos
- credenciais
- escrita em recursos persistentes

A AI Platform nunca deve assumir acesso irrestrito como padrão.

Quanto maior a capacidade de ação da IA, maior deve ser o rigor de controle.

10. Memória deve ser útil, limitada e governada

Memória não deve ser confundida com acúmulo irrestrito de histórico.

O GrowthBot DE deve diferenciar claramente:

- contexto transitório da conversa atual
- estado persistido necessário para continuidade operacional
- sinais históricos relevantes para personalização futura

Memória só deve ser mantida quando gerar valor claro para:

- continuidade da jornada
- melhoria de recomendação
- automação posterior
- operação e análise

Persistir tudo indiscriminadamente aumenta custo, ruído, risco e dificuldade de governança.

11. Agentes devem ser introduzidos com disciplina arquitetural

Nem todo fluxo de IA precisa ser modelado como agente.

O projeto deve preferir fluxos determinísticos e explícitos sempre que eles resolverem bem o problema.

Agentes só devem ser introduzidos quando houver necessidade real de:

- decomposição de tarefa
- coordenação entre capacidades distintas
- uso iterativo de ferramentas
- raciocínio operacional multi-etapas
- execução controlada de subtarefas

Antes de criar um agente, deve estar claro:

- qual papel ele exerce
- quais entradas recebe
- quais ferramentas pode usar
- quais saídas pode produzir
- quais limites operacionais possui
- como seu comportamento será observado

Agentes sem fronteira clara se tornam fontes de opacidade e instabilidade.

12. O sistema deve preferir determinismo onde ele for viável

A camada de IA não deve introduzir não determinismo em partes do sistema que podem ser resolvidas com regras claras, dados conhecidos ou lógica de negócio explícita.

Modelos devem ser usados para problemas que realmente se beneficiam de inferência probabilística, como interpretação, sumarização, classificação contextual ou geração controlada.

Não usar LLM como substituto de:

- lookup simples
- regras de negócio estáveis
- mapeamentos explícitos
- validações estruturais
- decisões binárias totalmente conhecidas

IA deve aumentar inteligência, não substituir disciplina de engenharia.

13. Fallbacks são obrigatórios em fluxos críticos

Falhas de IA devem ser tratadas como cenário esperado, e não como exceção improvável.

A AI Platform deve prever estratégias de fallback para situações como:

- timeout
- resposta inválida
- modelo indisponível
- tool indisponível
- MCP indisponível
- contexto insuficiente
- custo ou latência acima do aceitável

Fallback não significa esconder erro silenciosamente.

Significa preservar comportamento seguro, continuidade razoável e rastreabilidade do que ocorreu.

14. Custo, latência e qualidade são trade-offs explícitos

Toda decisão sobre uso de modelos deve considerar ao mesmo tempo:

- qualidade da resposta
- latência
- custo por inferência
- previsibilidade operacional
- impacto no funil comercial

O melhor modelo técnico nem sempre será a melhor escolha operacional.

A AI Platform deve permitir decisões diferentes para contextos diferentes, como:

- classificação rápida
- recomendação estruturada
- geração mais sofisticada
- tarefas offline
- automações assíncronas

Esses trade-offs devem ser conscientes e revisáveis.

15. Respostas de IA devem ser rastreáveis

Sempre que uma resposta de IA influenciar o comportamento do sistema, deve ser possível reconstruir minimamente:

- qual fluxo a gerou
- qual contexto principal foi usado
- qual modelo foi acionado
- qual ferramenta ou MCP participou
- qual saída foi validada
- qual decisão operacional resultou disso

Rastreabilidade não exige registrar conteúdo sensível sem critério.

Exige registrar informação suficiente para auditoria, depuração, melhoria contínua e análise de qualidade.

16. A AI Platform deve ser observável

Observabilidade da camada de IA não é opcional.

No mínimo, a plataforma deve evoluir para medir:

- volume de inferências
- taxa de erro
- taxa de respostas inválidas
- uso de ferramentas
- uso de MCP Servers
- latência por fluxo
- custo estimado por fluxo
- impacto em eventos comerciais relevantes

Sem isso, a IA vira uma caixa-preta cara e difícil de otimizar.

17. Catálogo, contexto e recomendação não devem colapsar em uma única decisão opaca

O GrowthBot DE deve preservar separação entre:

- interpretação de intenção
- seleção de candidatos
- composição de contexto
- recomendação final

Mesmo quando a IA participar de mais de uma dessas etapas, o sistema deve evitar colapsar tudo em uma única chamada opaca sem visibilidade intermediária.

Separar etapas melhora:

- auditabilidade
- testabilidade
- controle de qualidade
- capacidade de evolução

18. A IA deve respeitar a filosofia comercial do produto

O GrowthBot DE não existe para empurrar qualquer produto.

A AI Platform deve ser orientada para recomendar a solução mais adequada dentro dos limites do catálogo, do contexto disponível e da confiança possível da inferência.

O sistema nunca deve ser projetado para maximizar comissão de forma cega às custas de:

- necessidade real do usuário
- qualidade da recomendação
- consistência da conversa
- confiança construída
- integridade do produto

Receita é consequência de confiança operacionalizada corretamente.

19. Evolução da plataforma deve ser incremental

A AI Platform deve ser capaz de evoluir por estágios, sem saltos desnecessários de complexidade.

Uma trajetória saudável pode incluir:

- chamadas simples a modelo
- prompts estruturados

- validação de saída
- uso controlado de ferramentas
- integração com MCP Servers
- memória governada
- agentes especializados
- coordenação multiagente, se um dia for justificável

Não antecipar toda complexidade futura na primeira versão.

Também não bloquear o futuro por excesso de simplicidade estrutural.

20. Toda mudança na camada de IA deve responder três perguntas

Antes de expandir a AI Platform, deve estar claro:

1. qual valor real essa mudança gera para o produto?
2. qual risco estrutural ou operacional ela introduz?
3. como essa mudança será medida, validada e mantida?

Se essas respostas não estiverem claras, a mudança ainda não está madura para entrar.

AI Decision Framework

Antes de utilizar IA em qualquer fluxo novo ou em qualquer expansão relevante da plataforma, a decisão deve passar por um framework explícito.

No mínimo, as seguintes perguntas devem ser respondidas:

1. este problema realmente exige IA?
2. existe uma solução determinística suficiente?
3. existe uma ferramenta mais apropriada?
4. existe um MCP adequado para enriquecer o fluxo?
5. qual modelo possui melhor relação entre custo, latência e qualidade?
6. existe risco relevante de alucinação?
7. existe risco operacional relevante?
8. como a resposta será validada?
9. como esse fluxo será monitorado?

Esse framework existe para impedir o uso impulsivo de IA.

A decisão correta, em muitos casos, será não usar IA.

AI Anti-Patterns

A AI Platform deve evitar explicitamente padrões que degradam qualidade, previsibilidade e governança.

Nunca:

- usar LLM como banco de dados
- usar LLM para validações simples
- usar prompts enormes sem necessidade real
- misturar contexto permanente com contexto transitório sem controle
- criar agentes sem responsabilidade clara
- usar IA onde regras explícitas resolvem melhor
- usar IA apenas porque ela existe
- delegar decisões operacionais críticas a saídas não validadas
- esconder fragilidade arquitetural atrás de prompts ou tools

Esses anti-patterns devem ser tratados como sinais de alerta estrutural.

Quando surgirem, o problema geralmente não está no modelo em si, mas na forma como a camada de IA está sendo desenhada.

Future Compatibility

A AI Platform deve permanecer preparada para evoluir sem bloqueios prematuros.

Isso inclui compatibilidade conceitual com cenários futuros como:

- OpenAI
- Claude
- Gemini
- Mistral
- Llama
- modelos locais
- RAG
- embeddings
- memory layers
- knowledge graphs
- MCP
- planning agents
- voice interfaces
- vision capabilities
- realtime interactions

Preparação para compatibilidade futura não significa implementar tudo agora.

Significa não tomar decisões estruturais que impeçam essas evoluções caso elas passem a fazer sentido para o produto.

Definition of Done

Uma alteração na AI Platform só pode ser considerada concluída quando atender, de forma proporcional ao seu escopo, aos seguintes critérios:

- segue a missão do produto
- respeita a arquitetura definida
- possui validação adequada
- possui fallback compatível com o risco
- possui observabilidade mínima
- possui documentação suficiente
- possui testes compatíveis com o impacto
- possui rastreabilidade operacional
- mantém baixo acoplamento estrutural
- possui custo justificável

Sem esses critérios, a mudança pode até funcionar tecnicamente, mas ainda não está pronta para ser tratada como parte estável da plataforma.

Checklist

- O uso de IA neste fluxo resolve um problema que realmente exige inferência?
- O modelo está isolado o suficiente para futura substituição?
- O contexto enviado está estruturado e minimamente controlado?
- O prompt possui objetivo claro e comportamento esperado explícito?
- A saída do modelo passa por validação antes de produzir efeito operacional?
- O uso de tool ou MCP possui propósito claro e privilégio mínimo?
- Existe fallback para falha, timeout ou resposta inválida?
- O fluxo é rastreável e observável?
- A decisão respeita a filosofia comercial e a missão do produto?

- A complexidade introduzida é proporcional ao valor gerado?
- O AI Decision Framework foi respondido com clareza antes da implementação?
- A mudança evita os anti-patterns já conhecidos da plataforma?
- A alteração preserva compatibilidade futura sem antecipar complexidade desnecessária?
- A mudança atende ao Definition of Done da AI Platform?

Observações

Este documento deve permanecer válido mesmo com a evolução de modelos, provedores e ferramentas.

A tecnologia da camada de IA mudará ao longo do tempo. Os princípios de governança, modularidade, validação, rastreabilidade e responsabilidade operacional não devem mudar com a mesma facilidade.

Sempre que houver tensão entre mais capacidade de IA e menos controle estrutural, o GrowthBot DE deve preferir controle estrutural.

Backend API

Backend API

Este documento define a política oficial de Backend and API Engineering do GrowthBot DE. Ele estabelece como o backend deve organizar fronteiras, contratos, validações, persistência, integrações e comportamento operacional para sustentar uma AI Sales Platform previsível, segura, testável e evolutiva.

Objetivo

Definir as regras obrigatórias que governam o backend e as APIs do GrowthBot DE.

O objetivo deste documento é impedir que o backend evolua como um conjunto difuso de handlers, integrações e acessos a dados sem fronteiras claras.

Backend and API Engineering, neste projeto, deve garantir que o sistema:

- tenha limites explícitos entre camadas
- preserve coerência entre produto e implementação
- mantenha contratos previsíveis
- trate erros com disciplina
- seja compatível com segurança, testes e observabilidade
- suporte evolução futura sem colapso estrutural

O backend deve ser tratado como fronteira crítica entre interface, domínio, IA, integrações externas e persistência.

Escopo

Este documento cobre:

- boundaries entre rotas, handlers, services, use cases e repositories
- validação de entradas e saídas
- contratos internos e externos
- classificação e tratamento de erros
- autenticação e autorização no servidor
- idempotência
- paginação, filtros e ordenação
- persistência e transações
- integrações externas
- webhooks
- comunicação com a AI Platform
- eventos de domínio
- logs, métricas e correlation IDs
- timeouts, retries e fallbacks
- versionamento e compatibilidade
- separação entre negócio e infraestrutura
- preparação para banco compartilhado, filas, workers e múltiplos runtimes

Este documento não redefine a missão do produto, não substitui a arquitetura geral e não detalha a política de segurança, testes ou observabilidade. Ele traduz essas diretrizes para o comportamento específico do backend e das APIs.

Regras

1. O backend deve ser tratado como fronteira de controle

O backend do GrowthBot DE não é apenas camada de passagem entre request e database.

Ele é a fronteira que deve:

- validar entradas
- proteger regras de negócio
- coordenar casos de uso
- limitar integrações externas
- preservar contratos
- produzir rastreabilidade operacional

Sempre que uma requisição entra no sistema, o backend deve convertê-la em comportamento controlado e previsível.

2. Fronteiras entre camadas devem ser explícitas

O sistema deve preservar separação clara entre:

- rotas
- handlers
- services
- use cases
- repositories
- integrações externas

Cada camada deve existir com responsabilidade definida.

Misturar parsing de request, regra de negócio, acesso a banco, chamada externa e montagem de resposta no mesmo ponto do sistema deve ser tratado como degradação arquitetural.

3. Rotas devem ser finas

Rotas devem existir para:

- receber requests
- delegar ao handler ou caso de uso apropriado
- devolver resposta compatível com o contrato

Rotas não devem concentrar:

- regra de negócio relevante
- acesso direto a banco
- chamadas diretas complexas para integrações externas
- lógica condicional extensa

Quanto mais lógica de domínio existe na rota, mais frágil a fronteira do backend se torna.

4. Handlers devem orquestrar, não dominar o sistema

Handlers podem coordenar parsing, autorização contextual, chamada de caso de uso e normalização de resposta.

Mas não devem se transformar em centro de toda a lógica do backend.

Se o handler passa a decidir regras de negócio, persistência, retry, fallback, mapping e observabilidade profunda sozinho, a responsabilidade foi violada.

5. Use cases devem concentrar comportamento de negócio

Casos de uso devem representar intenções relevantes do sistema, como:

- gerar recomendação
- registrar clique
- agendar follow-up
- consultar visão operacional

Use cases devem conter a coordenação principal do comportamento de domínio.

Eles não devem depender de detalhes de framework ou de transporte HTTP.

6. Services não devem substituir modelagem de domínio

Services podem existir para encapsular lógica compartilhada, colaboração entre dependências ou coordenação operacional.

Mas não devem virar categoria genérica onde qualquer coisa sem lugar definido é colocada.

Service é conceito válido apenas quando sua responsabilidade é clara.

7. Repositories devem abstrair persistência, não negócio

Repositories existem para lidar com leitura e escrita de dados.

Eles não devem:

- decidir regra de negócio
- validar semântica principal de domínio
- conhecer detalhes de UI
- coordenar fluxos de produto

Repository deve proteger a separação entre domínio e mecanismo de persistência.

8. Entradas devem ser validadas na borda

Toda entrada externa deve ser validada antes de produzir efeito relevante no sistema.

Isso inclui:

- parâmetros de rota
- query params
- payloads
- headers relevantes
- dados vindos de webhooks
- dados vindos de integrações

Não confiar em input externo apenas porque ele vem de frontend próprio, Telegram, dashboard interno ou parceiro conhecido.

9. Saídas também devem ser controladas

O backend não deve tratar resposta como detalhe espontâneo da implementação.

Saídas devem ser consistentes, previsíveis e alinhadas ao contrato esperado.

Isso vale tanto para respostas externas quanto para contratos internos entre módulos.

Retornar dados excessivos, inconsistentes ou mal normalizados enfraquece a estabilidade da plataforma.

10. Contratos internos e externos devem ser explícitos

Todo fluxo relevante deve operar sobre contratos compreensíveis.

Deve estar claro:

- o que entra
- o que sai
- o que é obrigatório
- o que é opcional
- o que representa falha

Contratos implícitos, inferidos informalmente pelo código, aumentam acoplamento e fragilidade.

11. O backend deve classificar erros com disciplina

Erros não devem ser tratados como massa homogênea.

O sistema deve distinguir, conceitualmente, entre:

- erro de validação

- erro de autenticação
- erro de autorização
- erro de domínio
- erro de integração externa
- erro de infraestrutura
- erro inesperado

Sem classificação, tratamento e observabilidade de erro ficam pobres.

12. Erros devem ser úteis, seguros e observáveis

Toda falha relevante deve produzir:

- comportamento seguro
- contexto suficiente para diagnóstico
- resposta adequada ao consumidor
- rastreabilidade operacional

Erro útil não significa erro verboso para o cliente.

Significa erro claro internamente e controlado externamente.

13. Autenticação e autorização devem existir no servidor

O backend não deve delegar confiança apenas ao cliente.

Toda ação sensível deve ser protegida no servidor por mecanismos compatíveis com o risco.

Além de autenticar quem faz a chamada, o sistema deve autorizar o que pode ser feito naquele contexto.

14. Idempotência deve ser tratada explicitamente em fluxos sensíveis

Operações sujeitas a repetição por retry, webhook, refresh, rede instável ou comportamento duplicado do cliente devem considerar idempotência.

Isso é especialmente importante para:

- registro de eventos
- agendamentos
- follow-ups
- integrações externas
- webhooks
- escritas críticas

Se uma repetição puder causar estado incorreto, o fluxo precisa de proteção explícita.

15. Paginação, filtros e ordenação devem ser estáveis

Sempre que o backend expuser coleções relevantes, o comportamento de paginação, filtragem e ordenação deve ser previsível.

Não permitir endpoints que cresçam sem limites claros ou retornem listas arbitrárias sem critério de ordenação.

Coerência nesses contratos melhora:

- performance
- previsibilidade
- experiência do consumidor
- evolução futura

16. Persistência deve respeitar fronteiras transacionais

O backend deve tratar operações de escrita com clareza sobre:

- unidade de consistência
- momento da persistência

- dependências entre registros
- impacto de falhas parciais

Sempre que um fluxo depender de múltiplas alterações relacionadas, a consistência deve ser pensada explicitamente.

Transação não é detalhe tardio.

É decisão de integridade.

17. O backend não deve depender estruturalmente do banco atual

Mesmo que hoje a persistência use SQLite, o backend deve evoluir sem acoplamento irreversível a esse contexto local.

As decisões de modelagem devem considerar que o sistema poderá migrar para:

- banco compartilhado
- banco remoto
- banco com concorrência maior
- cenários multi-tenant

O backend deve proteger o domínio contra detalhes transitórios da infraestrutura atual.

18. Integrações externas devem ser encapsuladas

OpenAI, Telegram, MCP-related bridges, tracking targets e quaisquer serviços externos devem ser acessados por fronteiras explícitas.

O restante do sistema não deve depender diretamente de detalhes operacionais de cada integração.

Encapsulamento melhora:

- testabilidade
- segurança
- troca de provedor
- observabilidade
- fallback

19. Webhooks devem ser tratados como entrada hostil por padrão

Webhooks não devem ser considerados confiáveis apenas por existirem em integração conhecida.

Eles devem ser tratados com:

- validação
- autenticação ou verificação de origem quando aplicável
- idempotência
- controle de efeito colateral
- observabilidade

Webhook sem contenção é porta de inconsistência.

20. Comunicação com a AI Platform deve ser mediada

O backend não deve expor diretamente a complexidade da AI Platform para cada rota.

A comunicação com a camada de IA deve acontecer por contratos internos claros, com:

- contexto minimizado
- validação de saída
- fallback
- rastreabilidade

O backend deve proteger o restante da plataforma contra comportamento imprevisível da inferência.

21. Eventos de domínio devem ter semântica clara

Sempre que o backend produzir eventos internos ou operacionais, eles devem representar fatos relevantes do sistema.

Eventos não devem ser criados como efeito decorativo ou duplicação confusa de logs.

Deve estar claro:

- o que o evento significa
- quando ele ocorre
- quem o produz
- quem pode consumi-lo

22. Logs, métricas e correlation IDs são parte do backend

Backend não termina na resposta HTTP.

Ele também é responsável por produzir sinais úteis para:

- observabilidade
- auditoria
- análise de produto
- diagnóstico técnico

Todo fluxo relevante deve considerar logs estruturados, métricas coerentes e mecanismos de correlação quando necessário.

23. Timeouts, retries e fallbacks devem ser intencionais

Chamadas externas e fluxos sujeitos a falha não podem depender de esperança implícita.

Sempre que houver integração relevante, deve estar claro:

- quanto tempo esperar
- se retry faz sentido
- em que condição retry deve parar
- qual fallback existe
- qual impacto é aceitável

Retry sem critério amplifica problema.

Ausência total de fallback em fluxo crítico também.

24. Versionamento e compatibilidade devem ser pensados antes da ruptura

APIs, contratos internos e estruturas de resposta devem evoluir com disciplina.

Mesmo quando o sistema ainda é pequeno, o backend não deve ser tratado como território descartável sem preocupação de compatibilidade.

Mudanças que quebram consumidores ou fluxos internos precisam ser intencionais, justificadas e observáveis.

25. Separação entre negócio e infraestrutura é obrigatória

O backend deve proteger a regra de negócio contra acoplamento excessivo a:

- framework
- banco
- provider externo
- runtime específico
- detalhe de transporte

Quanto mais o domínio depende desses detalhes, mais cara fica a evolução futura.

26. O backend deve ser preparado para múltiplos runtimes

Mesmo sem implementar isso agora, a engenharia deve considerar que partes do sistema poderão rodar futuramente em:

- runtime web
- workers
- jobs assíncronos

- filas
- processos especializados

Backend bom não assume que tudo sempre acontecerá dentro do mesmo request-response síncrono.

27. O backend deve preservar compatibilidade com evolução operacional

Decisões de hoje não devem impedir amanhã:

- banco compartilhado
- filas
- workers
- múltiplos canais
- múltiplos tenants
- múltiplos runtimes
- integrações mais complexas

Isso não exige superarquitectura.

Exige não colapsar tudo em handlers frágeis e acoplados.

Backend Decision Framework

Antes de criar ou alterar comportamento relevante no backend, deve ser possível responder:

1. qual fronteira está sendo criada ou modificada?
2. em qual camada esse comportamento realmente pertence?
3. qual contrato de entrada e saída está sendo estabelecido?
4. qual validação precisa acontecer na borda?
5. qual regra de negócio está sendo protegida?
6. qual erro pode acontecer e como ele será classificado?
7. existe escrita ou operação que exige idempotência?
8. existe integração externa envolvida?
9. como esse fluxo será observado, medido e correlacionado?
10. essa decisão preserva evolução futura do backend?

Sem essas respostas, a mudança ainda não está madura.

Backend Anti-Patterns

O GrowthBot DE deve evitar explicitamente padrões que degradam a qualidade do backend.

Nunca:

- concentrar regra de negócio em rotas
- usar handlers como depósito genérico de lógica
- acessar banco diretamente de qualquer camada sem disciplina
- confiar em input sem validação
- misturar contratos externos e internos sem clareza
- tratar todos os erros da mesma forma
- acoplar domínio a framework ou provider específico
- usar retry sem critério
- aceitar efeitos críticos sem idempotência quando ela é necessária
- expor respostas instáveis ou excessivas por conveniência
- tratar webhook como entrada confiável por padrão
- esconder ausência de modelagem atrás de services genéricos

Esses anti-patterns devem ser tratados como sinais de erosão arquitetural.

Future Compatibility

O backend do GrowthBot DE deve permanecer preparado para evoluções futuras como:

- banco compartilhado
- PostgreSQL ou outras persistências mais robustas
- multi-tenant
- multi-language
- multi-channel
- filas e workers
- jobs assíncronos
- APIs públicas
- múltiplos runtimes
- parceiros externos
- integrações mais complexas
- coordenação com AI Platform mais sofisticada

Preparação para compatibilidade futura não significa antecipar toda complexidade agora.

Significa não construir um backend que dependa estruturalmente de um cenário único, local e frágil.

Definition of Done

Uma alteração relevante no backend ou nas APIs só pode ser considerada concluída quando, de forma proporcional ao seu escopo:

- respeita a missão do produto
- respeita a arquitetura
- pertence à camada correta
- possui contratos claros de entrada e saída
- valida inputs e controla outputs
- classifica erros de forma adequada
- protege autenticação e autorização quando necessário
- trata idempotência quando relevante
- respeita segurança, observabilidade e testes
- possui comportamento previsível diante de timeout, retry ou fallback
- evita anti-patterns conhecidos
- preserva compatibilidade futura razoável

Sem esses critérios, a mudança ainda não está madura para integrar o backend com confiança.

Checklist

- A responsabilidade está na camada correta?
- O contrato de entrada e saída está claro?
- Há validação suficiente na borda?
- O fluxo separa negócio de infraestrutura?
- Os erros relevantes estão classificados?
- Existe necessidade de autenticação ou autorização?
- Existe risco de repetição que exige idempotência?
- Há integração externa, timeout, retry ou fallback a considerar?
- O fluxo produz logs, métricas e correlation id quando necessário?
- A persistência respeita integridade e consistência?
- A mudança evita anti-patterns conhecidos?
- A decisão preserva evolução futura do backend?

- A entrega atende ao Definition of Done?

Observações

O backend do GrowthBot DE deve ser entendido como uma fronteira de estabilidade entre produto, IA, persistência, integrações e operação.

Seu papel não é apenas “fazer funcionar”.

Seu papel é fazer funcionar com previsibilidade, limites claros, segurança, observabilidade e capacidade de evolução.

Sempre que houver tensão entre conveniência imediata e disciplina estrutural do backend, a disciplina estrutural deve prevalecer.

Security

Security

Este documento define a política oficial de Security Engineering do GrowthBot DE. Ele estabelece como a plataforma deve proteger usuários, dados, integrações, agentes de IA, MCPs e infraestrutura ao longo de todo o ciclo de vida do produto.

Objetivo

Definir regras obrigatórias de segurança para o GrowthBot DE como AI Sales Platform.

O propósito deste documento não é listar boas práticas genéricas. Seu papel é estabelecer critérios de decisão, limites operacionais e padrões obrigatórios para reduzir risco, preservar confiança, proteger dados, restringir acessos e evitar que velocidade de desenvolvimento degrade a postura de segurança do projeto.

Security Engineering, neste contexto, não é uma camada isolada. É uma responsabilidade transversal que deve influenciar arquitetura, produto, IA, integrações, observabilidade, deploy, testes e operação diária.

Escopo

Este documento cobre:

- princípios de Security Engineering
- proteção de dados
- segredos e credenciais
- autenticação e autorização
- integrações externas
- OpenAI e provedores de IA
- Telegram Bot API
- MCP Servers e tools
- agentes de IA
- persistência e banco de dados
- logs, auditoria e rastreabilidade
- ambientes e deploy
- decisões de mudança com impacto em segurança
- anti-patterns de segurança
- Definition of Done para alterações sensíveis

Este documento não substitui a arquitetura do sistema, nem detalha implementação completa de infraestrutura. Ele define as regras que toda solução técnica deve respeitar para ser considerada aceitável do ponto de vista de segurança.

Regras

1. Segurança é requisito de produto e engenharia

No GrowthBot DE, segurança não é uma etapa final de revisão.

Ela deve estar presente desde a decisão de produto até a operação do sistema.

Toda funcionalidade que envolva:

- dados de usuário
- credenciais
- automação
- integrações externas
- IA

- MCPs
- tracking
- painéis operacionais

deve ser desenhada considerando risco desde o início.

Se segurança só é discutida depois da implementação, a engenharia já falhou.

2. O sistema deve operar com privilégio mínimo

Todo componente do GrowthBot DE deve receber apenas o nível mínimo de acesso necessário para cumprir sua função.

Isso vale para:

- variáveis de ambiente
- tokens
- banco de dados
- APIs externas
- ferramentas
- MCP Servers
- agentes
- acessos administrativos
- processos automatizados

A postura padrão deve ser restritiva.

Primeiro limitar. Depois ampliar apenas se houver necessidade real e justificada.

3. Segredos nunca devem ser tratados como configuração comum

Chaves, tokens, segredos, credenciais e identificadores sensíveis não devem ser:

- hardcoded em código
- versionados no repositório
- replicados em múltiplos arquivos sem necessidade
- expostos em logs
- enviados em prompts
- compartilhados entre ambientes sem controle

Segredos devem ser geridos fora do código versionado e com escopo claro de uso.

Placeholders podem existir no repositório.

Valores reais, nunca.

4. Ambientes devem ser isolados conceitualmente

Desenvolvimento, validação e produção não devem ser tratados como o mesmo ambiente com nomes diferentes.

Cada ambiente deve ter:

- segredos próprios
- tokens próprios
- dados apropriados ao seu contexto
- política de acesso proporcional ao risco

Reutilizar credenciais de produção em desenvolvimento é falha grave.

Misturar ambientes reduz rastreabilidade e amplia impacto de incidentes.

5. Dados devem ser coletados e retidos com critério

O GrowthBot DE não deve armazenar dados apenas porque eles podem ser úteis no futuro.

Toda coleta ou persistência deve ser justificável em termos de:

- necessidade operacional
- continuidade da jornada
- qualidade de recomendação
- rastreabilidade
- automação legítima
- análise de produto

Persistir mais dados do que o necessário amplia superfície de risco sem aumentar valor proporcional.

6. Minimização de dados é regra estrutural

Sempre que possível, o sistema deve preferir:

- menos dados persistidos
- menos campos sensíveis
- menos cópias do mesmo dado
- menos exposição de contexto entre módulos
- menos retenção de histórico irrelevante

Minimização de dados não é limitação funcional.

É mecanismo de proteção.

7. Dados sensíveis não devem circular sem necessidade

Dados de usuário, contexto conversacional, histórico, sinais comportamentais e credenciais não devem ser propagados livremente entre módulos, logs, ferramentas, dashboards e camadas de IA.

Antes de permitir circulação de um dado, deve estar claro:

- quem precisa desse dado
- por que precisa
- por quanto tempo
- em qual contexto
- com qual risco

Se a resposta não estiver clara, o dado não deve circular.

8. A camada de IA deve ser tratada como superfície de risco

Toda integração com modelos de IA cria riscos específicos, como:

- vazamento de contexto
- uso excessivo de dados
- inferência sobre informações sensíveis
- saídas incorretas ou inseguras
- automação indevida
- extrapolação não validada

Por isso, prompts, contexto, tools e respostas devem ser tratados também como elementos de segurança.

IA não é apenas tema de qualidade de resposta. É tema de risco operacional.

9. Nunca enviar mais contexto do que o necessário para inferência

Ao montar contexto para um modelo, o sistema deve enviar apenas o conjunto mínimo necessário para a tarefa.

Não incluir:

- segredos
- tokens
- credenciais
- dados internos irrelevantes

- informações sensíveis sem justificativa
- histórico excessivo por conveniência

Contexto enviado a modelos deve ser controlado, revisável e proporcional ao objetivo do fluxo.

10. Saídas de IA não podem executar efeitos críticos sem controle

Respostas de IA não devem produzir diretamente efeitos críticos sem validação intermediária adequada.

Isso vale para fluxos que afetem:

- recomendação operacional
- automação
- mensagens ao usuário
- tracking
- follow-up
- integrações
- acesso a dados
- uso de tools

O modelo pode sugerir.

O sistema é responsável por validar, limitar e decidir.

11. MCP Servers e tools devem ser tratados como extensões privilegiadas

Toda tool e todo MCP Server aumentam a superfície de ataque da plataforma.

Por isso, integrações desse tipo só devem ser permitidas quando houver:

- propósito claro
- ganho real
- escopo mínimo
- autenticação segura
- observabilidade
- rastreabilidade
- política de remoção ou substituição

Integrações com grande poder de leitura, escrita ou navegação exigem escrutínio adicional.

12. MCPs devem operar com autorização explícita e escopo mínimo

Nenhum MCP deve receber acesso amplo por padrão.

Cada MCP precisa ter claramente definido:

- o que pode ler
- o que pode escrever
- quais credenciais utiliza
- quais fluxos dependem dele
- como falhas são tratadas
- como seu uso é auditado

Se um MCP não puder ser usado de forma limitada, ele não está maduro para entrar na plataforma.

13. Agentes devem operar sob contenção

Agentes de IA não devem ter autonomia irrestrita.

Cada agente deve ter:

- papel definido
- ferramentas explicitamente autorizadas
- limites de ação

- critérios de validação
- rastreabilidade de decisões
- possibilidade de interrupção ou fallback

Quanto mais autonomia um agente possui, maior deve ser a exigência de controle.

14. Autenticação deve ser tratada como barreira real, não formalidade

Sempre que houver acesso administrativo, operacional ou sensível, mecanismos de autenticação devem ser proporcionais ao risco.

Não assumir que:

- ambiente interno é seguro por definição
- link obscuro é proteção suficiente
- ausência de UI pública reduz necessidade de controle

Acesso precisa ser controlado de forma explícita.

15. Autorização deve ser separada de autenticação

Saber quem é o ator não basta.

O sistema também deve controlar o que esse ator pode fazer.

Toda ação sensível deve considerar:

- identidade
- papel
- contexto
- escopo permitido

Não conceder permissões amplas apenas por simplicidade de implementação.

16. Banco de dados deve ser protegido como ativo operacional

Persistência não é apenas detalhe técnico.

O banco contém estado crítico da plataforma, incluindo:

- sessões
- histórico relevante
- follow-ups
- tracking
- métricas operacionais

Por isso, a camada de persistência deve seguir regras de:

- acesso mínimo
- separação de responsabilidades
- integridade de escrita
- proteção contra uso indevido
- cuidado com dados expostos em consultas, logs e dashboards

17. Integridade de dados vale tanto quanto confidencialidade

Segurança não significa apenas esconder dados.

Também significa impedir que estado crítico seja:

- corrompido
- sobrescrito sem critério
- duplicado sem controle
- alterado por fluxos errados
- produzido por saída inválida de IA

Uma plataforma comercial com dados incorretos também é uma plataforma insegura.

18. Logs e observabilidade não podem vazarem informação sensível

Logs devem ajudar investigação, não criar novos riscos.

Nunca registrar desnecessariamente:

- tokens
- segredos
- payloads sensíveis completos
- credenciais
- contexto excessivo do usuário
- conteúdo privado sem justificativa

Observabilidade precisa ser útil e segura ao mesmo tempo.

19. Toda ação sensível deve ser auditável

Mudanças com impacto operacional ou de segurança devem poder ser rastreadas minimamente.

Isso inclui, quando aplicável:

- origem da ação
- ator responsável
- fluxo que originou a mudança
- ferramenta utilizada
- horário
- resultado
- falha ou sucesso

Sem auditoria suficiente, incidentes ficam difíceis de entender e corrigir.

20. Segurança deve considerar falha como cenário normal

O sistema deve assumir que:

- integrações falham
- tokens expiram
- serviços externos ficam indisponíveis
- respostas de IA podem ser inválidas
- automações podem disparar em contexto incorreto
- dados podem chegar incompletos

Por isso, fluxos críticos devem possuir:

- fallback
- validação
- limitação de impacto
- rastreabilidade
- comportamento seguro por padrão

21. Secure by default é a postura obrigatória

O comportamento padrão da plataforma deve favorecer segurança, não conveniência.

Isso significa preferir:

- negar acesso por padrão
- exigir validação
- exigir justificativa para ampliar escopo
- manter defaults conservadores

- evitar exposição até haver necessidade real

Defaults inseguros multiplicam risco silenciosamente.

22. Segurança deve ser proporcional, mas nunca opcional

Nem todo fluxo exige o mesmo nível de proteção.

Mas nenhum fluxo relevante deve ficar sem avaliação de segurança só porque parece simples.

A profundidade do controle pode variar conforme:

- impacto
- exposição
- criticidade operacional
- tipo de dado
- capacidade de ação

O que não pode variar é a existência de disciplina.

Security Decision Framework

Antes de introduzir qualquer mudança com impacto em segurança, deve ser possível responder:

1. qual ativo está sendo protegido?
2. qual risco real existe?
3. quem pode ser impactado?
4. qual dado circula ou persiste?
5. qual integração externa está envolvida?
6. qual privilégio está sendo concedido?
7. existe alternativa mais simples e mais segura?
8. como o risco será reduzido?
9. como esse fluxo será monitorado?
10. como esse comportamento será validado e auditado?

Se essas perguntas não puderem ser respondidas, a decisão ainda não está madura.

Security Anti-Patterns

O GrowthBot DE deve evitar explicitamente padrões que fragilizam a postura de segurança.

Nunca:

- versionar segredos
- compartilhar credenciais entre ambientes
- conceder acesso amplo por conveniência
- enviar segredos ou contexto sensível para LLM sem necessidade
- usar MCP com escopo excessivo
- permitir agente agir sem limite claro
- registrar tokens ou payloads sensíveis em logs
- depender de obscuridade como mecanismo de proteção
- aceitar saída de IA sem validação em fluxo crítico
- persistir dados sem justificativa de valor e risco
- tratar segurança como etapa posterior ao desenvolvimento

Esses anti-patterns devem ser tratados como falhas de engenharia, não como atalhos aceitáveis.

Future Compatibility

A política de segurança deve permanecer preparada para a evolução futura da plataforma, incluindo cenários como:

- multi-tenant
- multi-language
- multi-channel
- novos provedores de IA
- novos MCP Servers
- novos parceiros
- SaaS
- APIs públicas
- filas e workers
- bancos mais complexos
- observabilidade distribuída
- agentes mais sofisticados

Preparação para compatibilidade futura não significa antecipar toda complexidade agora.

Significa não tomar decisões que inviabilizem controles adequados quando a plataforma crescer.

Definition of Done

Uma alteração com impacto em segurança só pode ser considerada concluída quando, de forma proporcional ao seu escopo:

- respeita a missão do produto
- respeita a arquitetura
- opera com privilégio mínimo
- protege segredos de forma adequada
- minimiza exposição de dados
- valida efeitos críticos antes de executá-los
- possui observabilidade compatível com o risco
- possui rastreabilidade e auditoria suficientes
- possui testes compatíveis com o impacto
- possui documentação suficiente
- evita anti-patterns conhecidos
- possui custo operacional justificável

Sem esses critérios, a mudança pode até funcionar, mas ainda não está madura para entrar com confiança no sistema.

Checklist

- O ativo sensível deste fluxo está claramente identificado?
- O risco principal foi explicitado?
- O acesso concedido é realmente o mínimo necessário?
- Há segredos, tokens ou credenciais envolvidos?
- Existe circulação desnecessária de dados?
- O contexto enviado para IA está minimizado?
- O uso de MCP ou tool está limitado e justificado?
- Há autenticação e autorização proporcionais ao risco?
- Existe validação antes de efeitos críticos?
- O fluxo possui fallback seguro?
- O comportamento é auditável?
- Os logs evitam exposição sensível?
- A mudança evita os anti-patterns de segurança?
- A compatibilidade futura foi preservada sem ampliar risco desnecessário?
- A entrega atende ao Definition of Done de Security Engineering?

Observações

Segurança deve ser lida como disciplina de engenharia contínua, não como documento reativo para momentos de incidente.

No GrowthBot DE, o objetivo de Security Engineering não é desacelerar o produto sem critério.

O objetivo é permitir que a plataforma cresça com confiança, limites claros e risco controlado.

Sempre que houver tensão entre conveniência de curto prazo e segurança estrutural, a segurança estrutural deve prevalecer.

Performance

Performance

Este documento define a política oficial de Performance Engineering do GrowthBot DE. Ele estabelece como a plataforma deve tratar latência, throughput, custo computacional, eficiência operacional e escalabilidade ao longo de sua evolução como AI Sales Platform.

Objetivo

Definir as regras obrigatórias que orientam decisões de performance no GrowthBot DE.

O objetivo deste documento não é otimizar tudo antecipadamente. O objetivo é impedir que o sistema cresça com gargalos previsíveis, custos descontrolados, latência degradante ou decisões arquiteturais que comprometam sua evolução operacional.

Performance Engineering, neste projeto, deve equilibrar:

- velocidade de resposta
- custo de execução
- estabilidade operacional
- previsibilidade de carga
- capacidade de escalar
- qualidade da experiência

Performance não é apenas eficiência técnica. Ela é parte da confiabilidade do produto e da sustentabilidade da plataforma.

Escopo

Este documento cobre:

- latência
- throughput
- uso de recursos
- custo computacional
- custo de inferência
- eficiência de queries
- eficiência de integração externa
- performance da AI Platform
- performance do backend
- performance de persistência
- paginação e volume de dados
- comportamento sob crescimento de carga
- performance observável
- critérios de decisão
- anti-patterns
- Definition of Done

Este documento não substitui Architecture, Backend API, Observability, Security, Testing ou AI Platform. Ele define como a plataforma deve manter desempenho sustentável sem sacrificar clareza, segurança ou capacidade de evolução.

Regras

1. Performance deve ser tratada como requisito de engenharia, não como reação tardia

No GrowthBot DE, performance não deve ser discutida apenas quando o sistema já está lento.

Toda funcionalidade relevante deve considerar:

- impacto em latência
- impacto em custo
- impacto em carga
- impacto em persistência
- impacto em integrações
- impacto em experiência do usuário ou operador

Isso não significa micro-otimizar tudo.

Significa não projetar com cegueira para gargalos óbvios.

2. O sistema deve otimizar os fluxos que realmente importam

Performance Engineering deve ser guiado por fluxos críticos, não por obsessão técnica descontextualizada.

No GrowthBot DE, atenção especial deve recair sobre:

- geração de recomendação
- classificação de intenção
- montagem de contexto
- tracking
- follow-up
- leitura operacional de dashboard
- integrações externas
- rotas com maior frequência ou maior impacto

O foco deve estar no que afeta a plataforma de verdade.

3. Performance deve ser orientada por medição

Nenhuma otimização relevante deve ser tratada como verdade apenas por intuição.

Antes de otimizar, deve ser possível responder:

- o que está lento?
- em qual etapa?
- com qual frequência?
- com qual impacto?
- em qual volume?

Sem medição, "otimização" vira suposição cara.

4. Latência deve ser tratada como parte da experiência

Tempo de resposta afeta:

- fluidez da conversa
- confiança do usuário
- capacidade operacional
- produtividade do operador
- percepção de qualidade

Latência não deve ser avaliada apenas como métrica técnica.

Ela é parte da experiência do produto.

5. Custo e performance devem ser pensados juntos

No GrowthBot DE, desempenho não pode ser analisado sem considerar custo.

Isso é especialmente importante para:

- inferências

- chamadas externas
- consultas repetidas
- automações
- operações de dashboard

A melhor performance técnica nem sempre justifica o custo.

O melhor custo nem sempre preserva a experiência necessária.

Performance Engineering deve equilibrar ambos conscientemente.

6. O sistema deve evitar trabalho desnecessário

Toda camada da plataforma deve evitar processamento, leitura, escrita, serialização, inferência ou tráfego sem motivo claro.

Sempre que possível, evitar:

- chamadas duplicadas
- recomputação desnecessária
- consultas excessivas
- payloads maiores do que o necessário
- processamento de contexto irrelevante
- logs excessivos sem propósito

Performance ruim frequentemente nasce de desperdício, não apenas de algoritmo ruim.

7. A AI Platform deve ser tratada como superfície de custo e latência

Inferência é uma das áreas com maior potencial de impacto em performance da plataforma.

Por isso, a camada de IA deve considerar explicitamente:

- tempo de inferência
- tamanho de contexto
- número de chamadas
- fallback entre modelos
- uso de tools
- uso de MCP Servers
- custo por execução

A AI Platform não pode ser uma zona sem governança de performance.

8. Mais contexto nem sempre significa melhor performance

Enviar mais contexto para o modelo pode:

- aumentar latência
- aumentar custo
- aumentar instabilidade
- reduzir previsibilidade

O sistema deve preferir contexto suficiente, e não contexto máximo.

Performance da IA depende tanto de qualidade de contexto quanto de quantidade.

9. Consultas e persistência devem crescer com disciplina

A camada de persistência deve evitar padrões que degradam com crescimento de volume.

É obrigatório pensar sobre:

- consultas repetidas
- leitura excessiva de registros
- ordenações caras sem necessidade
- retornos sem limite

- escrita redundante
- scans desnecessários

O fato de o volume atual ser pequeno não justifica decisões evidentemente frágeis.

10. Paginação, filtros e ordenação são também decisões de performance

Toda listagem relevante deve considerar:

- limite de volume
- critério claro de ordenação
- filtros úteis
- comportamento estável

Endpoints e consultas que retornam coleções sem limites são risco funcional e de performance.

11. Timeouts são proteção de performance

Esperar indefinidamente por integração, inferência ou operação crítica é comportamento estruturalmente ruim.

Timeouts devem ser definidos conforme o papel do fluxo.

Isso protege:

- usuário
- operador
- recursos do sistema
- previsibilidade operacional

Timeout ausente costuma transformar lentidão em indisponibilidade cascata.

12. Retries devem ser seletivos

Retry não deve ser reflexo automático.

Antes de adicionar retry, deve estar claro:

- qual falha pode ser transitória
- quantas tentativas fazem sentido
- qual intervalo é aceitável
- qual custo isso adiciona
- quando parar

Retry mal desenhado piora consumo, latência e impacto em dependências externas.

13. Fallback é parte da resiliência de performance

Quando performance ideal não for possível, o sistema deve preferir comportamento degradado mas útil, em vez de colapso total.

Isso pode incluir, conforme o fluxo:

- resposta simplificada
- modelo alternativo
- resultado parcial
- operação assíncrona futura
- fila para processamento posterior

Fallback bem pensado melhora resiliência sem esconder problema.

14. O backend deve preservar previsibilidade de carga

O backend do GrowthBot DE não deve ser desenhado apenas para funcionar em baixa carga local.

Suas decisões devem evitar:

- explosão de chamadas por request

- dependência síncrona excessiva
- acoplamento rígido entre etapas
- respostas desnecessariamente pesadas

Backend previsível é base para escala futura.

15. Dashboards devem ser operacionais e eficientes

Camadas administrativas e operacionais não devem depender de consultas ou agregações ingênuas que degradem com facilidade.

Dashboards devem ser construídos de forma a equilibrar:

- valor operacional
- custo de leitura
- atualidade dos dados
- simplicidade de manutenção

Painel útil não precisa recalcular tudo o tempo todo.

16. Processamento síncrono deve ser usado com critério

Nem toda tarefa precisa acontecer dentro do fluxo síncrono principal.

Sempre que o custo operacional ou a latência justificarem, o sistema deve estar preparado para evoluir parte do trabalho para:

- jobs
- filas
- workers
- processamento posterior

A arquitetura não deve presumir que toda lógica sempre caberá com segurança dentro do request-response imediato.

17. Performance deve preservar legibilidade e segurança estrutural

Otimização não pode justificar:

- violação de boundaries
- lógica obscura
- redução de segurança
- contratos frágeis
- comportamento difícil de testar

Performance Engineering não deve destruir Architecture para ganhar velocidade aparente.

18. Caching só deve existir quando fizer sentido real

Cache pode ser valioso, mas também aumenta:

- complexidade
- risco de inconsistência
- dificuldade de depuração

Antes de introduzir cache, deve estar claro:

- que problema ele resolve
- por quanto tempo o dado pode envelhecer
- como ocorre invalidação
- qual impacto de incoerência

Cache sem disciplina é fonte clássica de comportamento estranho.

19. Performance deve ser observável

O sistema deve evoluir com visibilidade suficiente sobre:

- latência por fluxo
- tempo de inferência
- custo estimado
- volume de chamadas
- tempo de consultas
- taxa de fallback
- degradação sob carga

Sem observabilidade, performance vira discussão subjetiva.

20. Testes e performance devem conversar

Mudanças com impacto relevante em performance devem ser acompanhadas, quando fizer sentido, por validação adequada.

Isso pode incluir:

- testes de comportamento sob volume
- testes de tempo relativo
- validação de regressão evidente
- cobertura de cenários de fallback

Performance Engineering não deve operar isolado de Testing.

Performance Decision Framework

Antes de introduzir mudança relevante com impacto em performance, deve ser possível responder:

1. qual fluxo precisa ser otimizado ou protegido?
2. qual problema real de latência, custo ou carga existe?
3. isso foi medido ou apenas suposto?
4. qual é a principal fonte de custo ou lentidão?
5. existe alternativa mais simples?
6. existe risco de aumentar complexidade sem ganho real?
7. qual impacto essa decisão terá em IA, backend, persistência ou dashboard?
8. como essa mudança será observada depois?
9. qual trade-off está sendo aceito?
10. essa otimização preserva evolução futura da plataforma?

Sem essas respostas, a mudança ainda não está madura.

Performance Anti-Patterns

O GrowthBot DE deve evitar explicitamente padrões que degradam eficiência e previsibilidade.

Nunca:

- otimizar sem medir
- micro-otimizar código irrelevante
- ignorar latência de integrações externas
- usar IA sem considerar custo e tempo
- retornar listas sem limites
- repetir chamadas e consultas desnecessárias
- adicionar retry indiscriminado
- introduzir cache sem política clara
- sacrificar arquitetura por otimização prematura
- confundir resposta rápida local com prontidão para escalar
- esconder gargalos por falta de observabilidade

Esses anti-patterns criam sistemas frágeis, caros e difíceis de evoluir.

Future Compatibility

A política de Performance Engineering deve permanecer preparada para cenários futuros como:

- banco compartilhado
- PostgreSQL ou outras persistências mais robustas
- multi-tenant
- multi-language
- multi-channel
- múltiplos modelos de IA
- múltiplos MCP Servers
- filas e workers
- jobs assíncronos
- APIs públicas
- serviços distribuídos
- dashboards mais complexos

Preparação para compatibilidade futura não significa antecipar toda complexidade agora.

Significa não projetar o sistema como se o contexto atual fosse o limite definitivo de crescimento.

Definition of Done

Uma alteração relevante só pode ser considerada concluída, do ponto de vista de performance, quando de forma proporcional ao seu escopo:

- considera latência, custo e carga
- protege fluxo realmente importante
- evita trabalho desnecessário
- respeita arquitetura e boundaries
- preserva segurança, testes e observabilidade
- trata timeout, retry ou fallback quando necessário
- evita anti-patterns conhecidos
- possui trade-offs compreendidos
- mantém compatibilidade futura razoável

Sem isso, a mudança ainda não está madura em termos de desempenho sustentável.

Checklist

- O fluxo crítico está claramente identificado?
- O problema de performance foi medido?
- Existe custo computacional ou de inferência relevante?
- Há trabalho desnecessário que pode ser removido?
- Há consultas, payloads ou chamadas excessivas?
- Timeout, retry e fallback foram considerados?
- O impacto em IA, backend e persistência está claro?
- A mudança preserva arquitetura, segurança e testabilidade?
- O comportamento está observável?
- A decisão evita anti-patterns conhecidos?
- A compatibilidade futura foi preservada?
- A entrega atende ao Definition of Done de Performance Engineering?

Observações

Performance no GrowthBot DE deve ser tratada como disciplina de sustentabilidade operacional.

O objetivo não é buscar velocidade máxima abstrata.

O objetivo é manter a plataforma responsiva, eficiente, previsível e economicamente saudável à medida que produto, IA, integrações e operação crescem.

Sempre que houver tensão entre conveniência imediata e performance estrutural sustentável, a performance estrutural sustentável deve prevalecer.

Testing

Testing

Este documento define a política oficial de Testing Engineering do GrowthBot DE. Ele estabelece como a plataforma deve validar comportamento, reduzir regressões, proteger fluxos críticos e sustentar evolução segura ao longo do tempo.

Objetivo

Definir as regras obrigatórias de testes para o GrowthBot DE como AI Sales Platform.

O objetivo deste documento não é maximizar quantidade de testes. O objetivo é garantir confiança proporcional ao risco.

Testing Engineering, neste projeto, deve:

- reduzir regressões
- validar comportamento crítico
- proteger a evolução do produto
- apoiar mudanças na camada de IA
- aumentar segurança de deploy
- permitir refatoração com controle

Testes devem ser tratados como parte do sistema de engenharia, não como atividade acessória ou ritual de qualidade.

Escopo

Este documento cobre:

- estratégia geral de testes
- critérios de priorização
- testes unitários
- testes de integração
- testes end-to-end
- testes para AI Platform
- testes para persistência e tracking
- mocks e fixtures
- determinismo
- cobertura orientada a risco
- critérios de decisão
- anti-patterns
- Definition of Done

Este documento não substitui Security, Observability, Architecture ou Product Engineering. Ele define como o sistema deve ser validado para que essas disciplinas possam evoluir com confiança.

Regras

1. Testes existem para proteger comportamento relevante

No GrowthBot DE, testes não devem ser escritos apenas para aumentar cobertura numérica ou satisfazer formalidade de processo.

Eles devem proteger:

- comportamento de negócio
- fluxos críticos

- decisões estruturais
- integrações relevantes
- evolução segura da plataforma

Se um teste não protege nada importante, seu valor deve ser questionado.

2. Cobertura deve ser orientada a risco

A plataforma não precisa do mesmo nível de teste em todas as áreas.

A prioridade deve recair sobre fluxos com maior impacto em:

- recomendação
- persistência
- tracking
- follow-up
- IA
- autenticação
- dashboard APIs
- integrações externas

Quanto maior o risco funcional, operacional ou comercial, maior deve ser a exigência de teste.

3. A estratégia de testes deve ser em camadas

O GrowthBot DE deve usar uma estratégia de validação composta por camadas com responsabilidades diferentes.

No mínimo:

- testes unitários para lógica isolada
- testes de integração para colaboração entre módulos
- testes end-to-end para fluxos reais da aplicação

Cada camada deve validar o que lhe cabe.

Não usar e2e para compensar ausência de testes unitários.

Não usar testes unitários para fingir que integrações complexas estão protegidas.

4. Testes unitários devem focar lógica e comportamento local

Testes unitários devem validar unidades coesas, rápidas e determinísticas.

Eles são especialmente adequados para:

- classificadores
- matchers
- formatadores
- validadores
- builders
- componentes visuais isolados
- regras puras de domínio

Esses testes devem ter setup mínimo e feedback rápido.

5. Testes de integração devem validar colaboração real entre partes do sistema

Testes de integração devem existir para validar fluxos onde o comportamento depende da interação entre:

- módulos
- persistência
- serviços
- repositórios
- camada de IA com abstrações controladas

- tracking
- automações

Integração não significa chamar tudo de verdade sem critério.

Significa validar a colaboração relevante com isolamento suficiente para manter previsibilidade.

6. End-to-end deve proteger jornadas críticas

Testes e2e devem ser reservados para jornadas importantes da aplicação.

No GrowthBot DE, isso inclui especialmente:

- carregamento do dashboard
- autenticação administrativa
- rotas críticas
- tracking de redirect
- fluxos principais de interface

E2E deve validar que o sistema funciona do ponto de vista do usuário ou operador.

Não deve ser a primeira linha de defesa para cada regra pequena.

7. A camada de IA deve ser testada de forma apropriada

A AI Platform não deve ser tratada como área impossível de testar.

Ela deve ser validada por meio de testes proporcionais ao seu comportamento, como:

- testes de construção de prompt
- testes de seleção de contexto
- testes de validação de saída
- testes de fallback
- testes de orquestração com respostas mockadas
- testes de integração com dependências substituídas de forma controlada

Não depender exclusivamente de chamadas reais a modelo para ter confiança na camada de IA.

8. Testes devem preferir comportamento observável a implementação interna

O foco do teste deve ser o que o sistema faz, não como ele está implementado internamente.

Evitar testes que:

- travam refatoração sem ganho real
- acoplam a detalhes internos
- verificam estrutura privada sem necessidade
- duplicam a implementação em forma de expectativa

Quanto mais o teste depende de detalhe interno, menor seu valor de longo prazo.

9. Determinismo é requisito importante

Testes devem ser previsíveis.

Sempre que possível, controlar:

- tempo
- banco de teste
- dados de entrada
- respostas externas mockadas
- ordem de execução relevante

Teste que passa ou falha de forma instável é custo, não proteção.

10. Banco de dados de produção nunca deve ser usado em testes

Testes devem usar ambiente isolado.

Persistência em teste deve operar com:

- banco próprio de teste
- dados controlados
- limpeza explícita
- independência entre execuções

Misturar teste com estado real compromete confiança e segurança.

11. Mocks devem ser pequenos, focados e honestos

Mocks devem existir para isolar dependências ou controlar cenários específicos.

Eles não devem:

- replicar metade do sistema real
- esconder comportamento importante
- simular cenários irreais sem motivo
- introduzir mais complexidade do que a dependência original

Mock bom é o menor mock que preserva intenção do teste.

12. Fixtures devem representar domínio, não lixo de teste

Fixtures devem ser:

- pequenas
- legíveis
- reutilizáveis quando faz sentido
- próximas do domínio real

Evitar grandes blocos de dados genéricos que ninguém entende e ninguém confia.

Fixtures devem tornar o comportamento do teste mais claro.

13. Cada teste deve ter propósito explícito

Antes de adicionar um teste, deve estar claro:

- qual comportamento ele protege
- por que esse comportamento importa
- qual regressão ele evitará

Teste sem propósito explícito tende a virar peso morto.

14. O sistema deve testar tanto sucesso quanto falha

Fluxos críticos não devem ser testados apenas no cenário feliz.

Sempre que relevante, devem existir testes para:

- falha de integração
- resposta inválida
- ausência de dado esperado
- fallback acionado
- persistência inconsistente
- fluxo interrompido

Uma plataforma robusta precisa provar também como falha com segurança.

15. Segurança, observabilidade e testes devem conversar

Mudanças sensíveis devem ser validadas também sob perspectivas como:

- vazamento indevido de dados

- comportamento de fallback
- preservação de logs úteis
- bloqueio de fluxos inseguros

Testing Engineering não pode ignorar requisitos de Security e Observability.

16. Testes devem apoiar refatoração e não impedi-la artificialmente

Boa suíte de testes permite melhorar arquitetura sem medo excessivo.

Má suíte de testes:

- quebra por detalhe irrelevante
- exige manutenção excessiva
- desincentiva mudança saudável

O objetivo é proteger comportamento, não congelar implementação.

17. Cobertura numérica não deve dirigir sozinha a estratégia

Percentual de cobertura pode ser útil como sinal.

Mas não deve ser tratado como prova suficiente de qualidade.

Cobertura alta com testes fracos continua sendo proteção fraca.

Cobertura menor, mas concentrada nos fluxos críticos, costuma ser mais valiosa que cobertura ampla e superficial.

18. Testes devem acompanhar a evolução do handbook

À medida que Architecture, AI Platform, Security, Product Engineering e Observability amadurecem, a estratégia de testes também deve amadurecer.

Isso significa que testes precisam refletir:

- novos riscos
- novas integrações
- novas jornadas
- novas garantias esperadas

Testing não é estático.

19. Testes devem existir antes de promover mudança crítica

Mudanças com impacto relevante não devem depender apenas de inspeção manual antes de promoção entre branches.

Quando o risco justificar, a alteração deve estar protegida por testes compatíveis com seu impacto antes de ser promovida.

Revisão humana é necessária.

Sozinha, não é suficiente.

20. O custo dos testes também deve ser considerado

Toda suíte tem custo de:

- escrita
- manutenção
- execução
- diagnóstico

Por isso, a estratégia correta não é testar tudo.

É testar o que realmente merece proteção com a técnica certa.

Testing Decision Framework

Antes de adicionar, alterar ou deixar de criar um teste relevante, deve ser possível responder:

1. qual comportamento precisa ser protegido?
2. qual risco existe se isso quebrar?
3. qual camada de teste é mais apropriada?
4. existe forma mais simples de validar esse comportamento?
5. o teste será determinístico?
6. há dependência externa que precisa ser isolada?
7. o valor do teste justifica seu custo de manutenção?
8. esse teste protege comportamento ou apenas implementação?
9. existe cenário de falha que também deve ser coberto?
10. esse teste melhora realmente a confiança do projeto?

Sem essas respostas, o teste ou a ausência dele ainda não estão bem justificados.

Testing Anti-Patterns

O GrowthBot DE deve evitar explicitamente padrões que enfraquecem a estratégia de testes.

Nunca:

- criar testes só para inflar cobertura
- testar implementação interna sem motivo forte
- criar mocks gigantes
- usar banco real do projeto em testes
- depender de rede externa sem necessidade
- criar testes não determinísticos
- duplicar o mesmo teste em múltiplas camadas sem ganho real
- usar e2e para cobrir tudo
- ignorar cenários de falha em fluxos críticos
- escrever testes que ninguém entende
- aceitar suíte lenta e frágil como normal

Esses anti-patterns reduzem confiança e aumentam custo de evolução.

Future Compatibility

A política de Testing Engineering deve permanecer preparada para a evolução futura da plataforma, incluindo:

- multi-tenant
- multi-language
- multi-channel
- múltiplos provedores de IA
- múltiplos MCP Servers
- novos fluxos de automação
- autenticação mais robusta
- filas e workers
- banco mais complexo
- APIs públicas
- dashboards mais ricos

Preparação para compatibilidade futura não significa construir toda a suíte agora.

Significa não adotar uma estratégia de testes que colapse quando a plataforma crescer.

Definition of Done

Uma alteração relevante só pode ser considerada concluída, do ponto de vista de testes, quando de forma proporcional ao seu escopo:

- possui validação compatível com o risco
- protege comportamento relevante
- usa a camada de teste apropriada
- evita dependência desnecessária de detalhes internos
- cobre sucesso e falha quando relevante
- é determinística
- não usa estado real indevido
- possui mocks e fixtures legíveis quando necessários
- respeita segurança e observabilidade
- evita anti-patterns conhecidos
- possui custo de manutenção justificável

Sem isso, a mudança ainda não está suficientemente protegida.

Checklist

- O comportamento crítico foi identificado?
- O risco de regressão foi avaliado?
- A camada de teste escolhida é a correta?
- O teste é determinístico?
- Há dependências externas que precisam ser isoladas?
- Existe cenário de falha relevante coberto?
- O teste protege comportamento observável?
- Há uso indevido de banco, rede ou estado real?
- Mocks e fixtures estão claros e proporcionais?
- A mudança respeita Security e Observability?
- A estratégia evita anti-patterns conhecidos?
- A entrega atende ao Definition of Done de Testing Engineering?

Observações

Testing Engineering no GrowthBot DE deve ser lido como mecanismo de confiança contínua.

O objetivo não é provar perfeição.

O objetivo é reduzir incerteza suficiente para que a plataforma possa evoluir com velocidade, responsabilidade e controle.

Sempre que houver tensão entre conveniência de curto prazo e proteção real contra regressão, a proteção real deve prevalecer.

Frontend

Frontend

Este documento define a política oficial de Frontend Engineering do GrowthBot DE. Ele estabelece como interfaces, componentes, estados e fluxos visuais devem evoluir com clareza, consistência, acessibilidade e alinhamento com o produto.

Objetivo

Definir as regras obrigatórias que orientam o frontend do GrowthBot DE.

O objetivo deste documento é impedir que a interface evolua como coleção dispersa de componentes, estilos e estados sem coerência estrutural.

Escopo

Este documento cobre:

- organização de componentes
- boundaries entre UI, estado e lógica
- composição de interface
- acessibilidade
- consistência visual
- legibilidade
- estados de carregamento, erro e vazio
- integração com backend e AI Platform
- previsibilidade de UX
- testabilidade e observabilidade do frontend

Regras

1. O frontend deve servir clareza operacional e de produto

O GrowthBot DE não precisa de interface decorativa.

Ele precisa de interface clara, compreensível e consistente para:

- usuários
- operadores
- administração

2. Componentes devem ter responsabilidade clara

Cada componente deve existir com propósito compreensível.

Evitar componentes genéricos demais, grandes demais ou misturados com lógica que não pertence à UI.

3. A UI não deve concentrar regra de negócio indevida

O frontend pode conter lógica de apresentação, composição e estado local.

Regras centrais de domínio, persistência, autenticação crítica e decisões estruturais devem permanecer fora da camada visual.

4. Estados vazios, erro e carregamento são obrigatórios

Interfaces não devem assumir apenas cenário feliz.

Toda área relevante deve considerar:

- loading

- empty state
- error state
- degraded state quando aplicável

5. Consistência vale mais que criatividade isolada

O GrowthBot DE pode evoluir visualmente, mas a interface não deve parecer feita por partes desconectadas.

Texto, spacing, padrões de ação, feedback e estrutura devem permanecer coerentes.

6. Acessibilidade deve ser tratada como requisito de qualidade

Sem acessibilidade mínima, a interface perde robustez.

Semântica, foco, legibilidade e interação previsível devem ser considerados.

7. O frontend deve ser observável

Fluxos críticos do frontend devem produzir sinais úteis para operação e produto quando necessário.

8. O frontend deve respeitar contratos do backend

A UI não deve improvisar comportamento em cima de contratos incertos.

Integrações precisam ser explícitas e previsíveis.

9. Componentes devem favorecer reuso responsável

Reuso é valioso quando reduz duplicação real.

Não criar abstração visual prematura sem padrão comprovado.

10. A interface deve preservar opcionalidade futura

O frontend deve poder evoluir para mais fluxos, mais estados e mais superfícies sem colapsar por acoplamento excessivo.

Frontend Decision Framework

Antes de criar ou alterar componente ou fluxo relevante, deve ser possível responder:

1. qual problema de interface isso resolve?
2. isso melhora clareza do produto?
3. a responsabilidade está na camada correta?
4. existe estado de erro, loading e vazio?
5. isso respeita contratos do backend?
6. existe alternativa mais simples?
7. isso melhora consistência ou a piora?
8. a acessibilidade foi considerada?
9. a mudança é observável e testável quando necessário?
10. isso vale a complexidade criada?

Frontend Anti-Patterns

Nunca:

- colocar regra de negócio central em componente visual
- criar componente genérico sem necessidade
- ignorar estados de erro
- duplicar UI por conveniência
- violar contratos do backend por improviso
- aumentar complexidade visual sem ganho de clareza
- usar interface para esconder problema estrutural do sistema
- tratar acessibilidade como detalhe opcional

Future Compatibility

O frontend deve permanecer preparado para:

- novos painéis
- novos canais administrativos
- multi-language
- multi-tenant
- novos fluxos operacionais
- superfícies mais ricas de analytics e IA

Definition of Done

Uma alteração relevante no frontend só pode ser considerada concluída quando, de forma proporcional ao seu escopo:

- melhora clareza ou valor real do produto
- respeita boundaries corretos
- trata estados relevantes de interface
- mantém consistência visual e comportamental
- respeita contratos do backend
- é testável quando necessário
- considera acessibilidade
- evita anti-patterns conhecidos

Checklist

- A responsabilidade está na camada correta?
- A mudança melhora clareza?
- Estados de loading, erro e vazio foram tratados?
- Há acoplamento indevido com negócio?
- O contrato do backend foi respeitado?
- A acessibilidade foi considerada?
- A mudança mantém consistência?
- A entrega atende ao Definition of Done?

Observações

Frontend Engineering no GrowthBot DE deve servir entendimento, previsibilidade e operação.

Interface boa não é a que apenas parece moderna.

É a que deixa o sistema mais claro, confiável e utilizável.

Observability

Observability

Este documento define a política oficial de Observability Engineering do GrowthBot DE. Ele estabelece como a plataforma deve produzir visibilidade operacional, rastreabilidade, medição e capacidade de diagnóstico ao longo de todo o ciclo de vida do produto.

Objetivo

Definir as regras obrigatórias que tornam o GrowthBot DE observável como AI Sales Platform.

Observability, neste projeto, não existe apenas para investigar erros técnicos. Ela existe para tornar visíveis:

- comportamento do sistema
- fluxos do produto
- decisões da camada de IA
- eventos comerciais
- qualidade operacional
- gargalos estruturais
- impacto real das mudanças

O objetivo é impedir que a plataforma cresça como uma caixa-preta, onde há respostas, automações, cliques, follow-ups e recomendações, mas pouca capacidade de entender o que realmente está acontecendo.

Escopo

Este documento cobre:

- logs estruturados
- métricas técnicas e de produto
- tracing e correlação
- eventos do sistema
- eventos de IA
- eventos comerciais
- auditoria operacional
- diagnósticos de falha
- visibilidade da jornada
- dashboards
- alertabilidade futura
- critérios de decisão para introduzir observabilidade
- anti-patterns
- Definition of Done

Este documento não substitui Security Engineering, Testing, Product Engineering ou Architecture. Ele define como o sistema deve produzir visibilidade suficiente para que todas essas disciplinas operem com confiança.

Regras

1. Observabilidade é requisito estrutural, não luxo operacional

No GrowthBot DE, observabilidade não deve ser tratada como melhoria opcional para depois.

Toda funcionalidade relevante, especialmente quando envolver:

- IA

- tracking
- follow-up
- persistência
- integração externa
- automação
- dashboard
- jornada comercial

deve ser pensada com algum nível de observabilidade desde o início.

Se o sistema não consegue explicar o que fez, ele ainda não está maduro.

2. O sistema deve ser compreensível por fora

Observability Engineering existe para tornar o comportamento interno visível a partir de sinais externos confiáveis.

Isso significa que deve ser possível entender, com nível suficiente de detalhe:

- o que aconteceu
- quando aconteceu
- em qual fluxo aconteceu
- com qual resultado
- com qual impacto
- por qual componente passou

Sem isso, o GrowthBot DE não consegue evoluir de forma segura.

3. Logs devem ser estruturados

Logs não devem ser pensados como texto solto orientado apenas à leitura humana.

Logs relevantes devem ser estruturados para permitir:

- filtragem
- correlação
- agrupamento
- diagnóstico
- análise posterior

Sempre que fizer sentido, os logs devem conter campos previsíveis, como:

- timestamp
- componente
- tipo de evento
- status
- correlation id
- user id ou equivalente permitido
- flow id ou contexto operacional

Sem estrutura, log vira ruído.

4. Logs devem ser úteis e proporcionais

O sistema não deve registrar tudo indiscriminadamente.

Cada log deve existir porque ajuda a:

- entender comportamento
- investigar falhas
- acompanhar decisões relevantes
- reconstruir fluxo
- analisar impacto

Log excessivo aumenta custo, ruído e risco.

Log insuficiente impede investigação.

O objetivo é utilidade proporcional.

5. Segurança e observabilidade devem coexistir

Logs, traces e métricas não podem expor indevidamente:

- segredos
- tokens
- credenciais
- payloads sensíveis completos
- contexto excessivo de usuário
- informações privadas sem justificativa

Observabilidade não pode degradar a postura de segurança do sistema.

Toda visibilidade deve respeitar minimização e proteção de dados.

6. Correlation ID deve existir para fluxos relevantes

Todo fluxo operacional relevante deve ser rastreável por identificador de correlação ou mecanismo equivalente.

Isso é especialmente importante em fluxos com múltiplas etapas, como:

- entrada do usuário
- classificação de intenção
- seleção de candidatos
- inferência
- persistência
- clique
- follow-up
- atualização de dashboard

Sem correlação, cada log vira evento isolado e perde valor investigativo.

7. O sistema deve medir eventos, não apenas páginas e erros

GrowthBot DE não é apenas uma aplicação web.

Sua observabilidade deve se concentrar em eventos de produto e operação, como:

- usuário entrou em contato
- intenção foi classificada
- recomendação foi gerada
- produto foi selecionado
- clique foi registrado
- follow-up foi agendado
- follow-up foi enviado
- falha ocorreu em integração
- fluxo foi interrompido

Eventos são a linguagem observável do produto.

8. A jornada comercial deve ser observável ponta a ponta

O sistema deve evoluir para tornar mensurável o funil completo da plataforma.

No mínimo, deve haver capacidade futura de observar encadeamentos como:

usuário entrou

→ intenção foi detectada

→ recomendação foi gerada

→ clique ocorreu

→ follow-up foi enviado

→ re-engajamento ocorreu

→ resultado comercial foi medido

Se o sistema enxerga apenas partes isoladas da jornada, a otimização do produto fica comprometida.

9. IA deve ser observável como camada própria

A AI Platform precisa produzir sinais específicos de observabilidade.

No mínimo, a evolução da plataforma deve permitir medir:

- volume de inferências
- latência por fluxo
- taxa de erro
- taxa de resposta inválida
- uso de tools
- uso de MCP Servers
- fallback acionado
- custo estimado
- impacto operacional da inferência

IA sem observabilidade é uma caixa-preta cara.

10. Métricas técnicas e métricas de produto devem coexistir

O GrowthBot DE precisa combinar duas visões:

- saúde técnica do sistema
- impacto comportamental e comercial do produto

Não basta medir:

- uptime
- erros
- tempo de resposta

Também é necessário evoluir para medir:

- recomendação gerada
- clique por recomendação
- follow-up por segmento
- conversão por fluxo
- perda de usuários por etapa
- efetividade operacional

Uma plataforma comercial precisa das duas camadas.

11. Dashboards devem servir decisão, não ornamentação

Dashboards não devem existir apenas para mostrar números bonitos.

Cada painel deve responder a perguntas operacionais reais, como:

- o sistema está saudável?
- onde o fluxo quebra?
- a IA está ajudando ou atrapalhando?
- quais recomendações performam melhor?
- há gargalo em follow-up?

- há aumento de erro ou queda de conversão?

Se um dashboard não apoia decisão, ele provavelmente está mal definido.

12. Observabilidade deve apoiar depuração e também aprendizado

O objetivo não é apenas descobrir bugs.

Observabilidade deve permitir:

- investigar incidentes
- entender regressões
- validar hipóteses de produto
- revisar impacto de arquitetura
- otimizar IA
- ajustar automações
- melhorar jornada

Isso transforma visibilidade em capacidade real de evolução.

13. Métricas precisam de semântica clara

Nenhuma métrica importante deve existir sem definição clara.

Para cada métrica relevante, deve estar claro:

- o que ela representa
- quando ela aumenta
- quando ela diminui
- o que não está incluído nela
- por que ela importa

Métricas sem semântica consistente geram decisões ruins.

14. Nem todo sinal precisa virar métrica permanente

O sistema não deve transformar qualquer dado disponível em métrica obrigatória.

Uma métrica só deve existir se houver motivo claro para:

- acompanhar tendência
- detectar risco
- apoiar decisão
- validar hipótese
- revelar degradação

Métrica demais sem propósito produz fadiga e confusão.

15. Falhas devem ser observáveis com contexto suficiente

Quando um fluxo falhar, o sistema deve oferecer contexto suficiente para responder:

- onde falhou
- em qual etapa
- sob qual condição
- com qual insumo principal
- com qual impacto
- se houve fallback

Sem esse contexto, o tempo de diagnóstico cresce desnecessariamente.

16. Observabilidade deve cobrir integrações externas

Integrações como:

- OpenAI
- Telegram
- MCP Servers
- tools externas
- tracking redirects

devem gerar sinais observáveis próprios.

Não basta saber que o sistema falhou.

É necessário entender se a falha veio de dependência externa, da plataforma interna ou da interação entre ambas.

17. Auditoria e observabilidade se complementam

Nem todo evento observável é evento auditável.

Mas ações críticas devem ser registradas com cuidado suficiente para permitir reconstrução posterior.

Isso inclui, quando aplicável:

- alterações de estado
- envio de follow-up
- registro de clique
- resposta operacional relevante
- acionamento de fallback
- uso sensível de tool ou MCP

Sem isso, incidentes se tornam difíceis de investigar com precisão.

18. O sistema deve preferir eventos de domínio úteis

Além de logs técnicos, o GrowthBot DE deve evoluir para produzir eventos de domínio com significado operacional.

Exemplos conceituais:

- `intent_detected`
- `recommendation_generated`
- `click_registered`
- `followup_scheduled`
- `followup_sent`
- `ai_fallback_triggered`
- `catalog_match_failed`

Esses eventos aproximam engenharia, produto e operação.

19. Observabilidade deve ser incremental, mas intencional

Nem tudo precisa ser instrumentado com profundidade máxima na primeira versão.

Mas toda evolução importante deve perguntar:

- o que precisamos enxergar nesse fluxo?
- o que pode quebrar aqui?
- o que precisamos medir para melhorar isso depois?

Instrumentação pode crescer em etapas.

Falta de intenção não pode virar desculpa para cegueira estrutural.

20. Alertabilidade futura deve ser considerada

Mesmo quando o projeto ainda não possui alerting completo, as métricas e eventos devem ser desenhados de modo que futuramente possam sustentar:

- alertas de falha
- alertas de degradação

- alertas de latência
- alertas de custo
- alertas de volume anômalo
- alertas de conversão anormal

Observabilidade madura prepara o terreno para operações mais robustas.

Observability Decision Framework

Antes de adicionar ou alterar instrumentação relevante, deve ser possível responder:

1. qual fluxo estamos tentando tornar visível?
2. qual decisão essa visibilidade permitirá tomar?
3. qual evento, log, métrica ou trace realmente ajuda?
4. existe risco de excesso de ruído?
5. existe risco de exposição de dados?
6. qual correlação será necessária?
7. essa observabilidade serve depuração, produto, operação ou todos?
8. como essa informação será consumida?
9. qual custo de manter essa instrumentação?
10. essa visibilidade é proporcional ao valor e ao risco do fluxo?

Sem essas respostas, a instrumentação ainda não está madura.

Observability Anti-Patterns

O GrowthBot DE deve evitar explicitamente padrões que enfraquecem visibilidade real do sistema.

Nunca:

- logar tudo sem critério
- criar métricas sem semântica clara
- instrumentar apenas erros e ignorar fluxo de negócio
- expor dados sensíveis em logs ou dashboards
- medir volume sem medir qualidade
- gerar dashboards sem pergunta operacional real
- criar eventos duplicados e inconsistentes
- observar apenas frontend e ignorar backend ou IA
- registrar sinais sem correlation id quando o fluxo exige correlação
- confundir quantidade de dashboards com maturidade operacional

Esses anti-patterns geram falsa sensação de visibilidade.

Future Compatibility

A política de observabilidade deve permanecer preparada para a evolução futura da plataforma, incluindo:

- multi-tenant
- multi-language
- multi-channel
- múltiplos modelos de IA
- múltiplos MCP Servers
- novos parceiros
- filas e workers
- serviços distribuídos
- bancos mais robustos
- tracing distribuído

- APIs públicas
- automações mais sofisticadas

Preparação para compatibilidade futura não significa antecipar toda a complexidade.

Significa não criar instrumentação que só funcione em um cenário pequeno e frágil demais.

Definition of Done

Uma alteração relevante só pode ser considerada concluída, do ponto de vista de observabilidade, quando de forma proporcional ao seu escopo:

- torna o fluxo principal visível
- registra sinais úteis e não apenas ruído
- respeita segurança e minimização de dados
- possui logs estruturados quando necessário
- possui correlação suficiente quando o fluxo exige
- mede impacto técnico ou de produto de forma coerente
- permite diagnóstico razoável em caso de falha
- produz eventos com semântica clara
- possui documentação suficiente
- evita anti-patterns conhecidos
- preserva compatibilidade futura relevante

Sem isso, a mudança pode funcionar, mas continua difícil de operar e evoluir.

Checklist

- O fluxo relevante está observável?
- Está claro qual decisão essa instrumentação apoia?
- Logs estão estruturados quando necessário?
- Há risco de ruído excessivo?
- Há risco de exposição sensível?
- Existe correlação id ou mecanismo equivalente quando necessário?
- Eventos de domínio foram considerados?
- Métricas possuem semântica clara?
- O fluxo de IA está observável?
- O impacto comercial ou operacional está mensurável?
- A instrumentação evita anti-patterns conhecidos?
- A compatibilidade futura foi preservada?
- A entrega atende ao Definition of Done de Observability Engineering?

Observações

Observabilidade no GrowthBot DE deve servir simultaneamente engenharia, produto, IA e operação.

Isso significa que o sistema precisa ser observável não apenas como software, mas como plataforma comercial orientada por conversas, decisões e automações.

Sempre que houver tensão entre conveniência de implementação e visibilidade estrutural mínima, a visibilidade estrutural deve prevalecer.

Deployment

Deployment

Este documento define a política oficial de Deployment Engineering do GrowthBot DE. Ele estabelece como mudanças devem ser promovidas entre branches, ambientes e versões com segurança, previsibilidade, rastreabilidade e controle operacional.

Objetivo

Definir as regras obrigatórias de deploy, promoção, rollout, rollback e gestão de ambientes do GrowthBot DE.

O objetivo deste documento é impedir que publicação de software seja tratada como ato informal, manual e pouco rastreável.

Deployment Engineering, neste projeto, deve garantir que a plataforma evolua com:

- segurança operacional
- separação clara entre ambientes
- rollout intencional
- rollback possível
- proteção de segredos
- controle de migrations
- compatibilidade com crescimento futuro

Escopo

Este documento cobre:

- fluxo entre `develop` e `main`
- relação entre branches e ambientes
- critérios de promoção
- gestão de ambientes
- configuração e segredos
- migrations
- rollout e rollback
- validação pós-deploy
- observabilidade de release
- compatibilidade futura com CI/CD e múltiplos ambientes

Este documento não substitui Security, Testing ou Observability. Ele traduz essas disciplinas para o momento de publicação e operação da plataforma.

Regras

1. O fluxo oficial continua sendo `develop -> main`

Toda mudança deve entrar primeiro em `develop`.

`main` deve receber apenas alterações promovidas de forma intencional, após validação suficiente.

Esse fluxo é obrigatório porque reduz risco, preserva estabilidade narrativa do projeto e evita que produção se torne ambiente de experimentação.

2. Branch não é ambiente, mas influencia ambiente

Mesmo que a topologia de ambientes evolua no futuro, a relação conceitual deve permanecer clara:

- `develop` concentra integração e validação contínua
- `main` representa o estado mais estável e promovível do sistema

Não assumir que uma branch substitui por si só a disciplina de ambiente.

3. Ambientes devem ter propósito explícito

Cada ambiente futuro da plataforma deve existir com função clara, como:

- desenvolvimento
- validação
- staging
- produção

Ambientes sem propósito explícito geram confusão operacional.

4. Segredos e configuração devem ser isolados por ambiente

Cada ambiente deve possuir:

- segredos próprios
- tokens próprios
- configuração própria
- acesso proporcional ao risco

Reutilizar segredos de produção em ambientes inferiores é falha grave.

5. Deploy deve ser rastreável

Toda promoção relevante deve permitir reconstruir:

- o que foi promovido
- quando foi promovido
- por quem foi promovido
- de qual branch ou estado veio
- com qual objetivo
- com qual resultado

Sem rastreabilidade, rollback e investigação ficam frágeis.

6. Nenhum deploy deve depender de estado implícito

Publicação não deve depender de conhecimento informal, memória individual ou sequência escondida de passos.

O processo de deploy deve ser compreensível, repetível e suficientemente documentado.

7. Rollout deve ser intencional

Toda publicação precisa considerar:

- escopo da mudança
- risco da mudança
- impacto operacional
- dependências externas
- necessidade de monitoramento reforçado

Deploy não é apenas “subir código”. É promover comportamento novo no sistema.

8. Rollback deve ser possível e pensado antes

Antes de promover mudança relevante, deve estar claro:

- como voltar atrás
- o que é reversível
- o que não é reversível facilmente
- se há mudança de dados envolvida
- qual plano existe em caso de degradação

Rollback não pode ser improvisado.

9. Migrations devem ser tratadas como mudança crítica

Mudanças de esquema, dados ou persistência exigem cuidado adicional.

Toda migration deve considerar:

- compatibilidade com a versão publicada
- impacto em leitura e escrita
- risco de lock ou indisponibilidade
- possibilidade de rollback
- integridade de dados

Migration nunca deve ser tratada como detalhe secundário do deploy.

10. Deploy deve respeitar Security, Testing e Observability

Nenhuma promoção relevante deve ignorar:

- cobertura ou validação compatível com risco
- proteção de segredos
- logs, métricas e sinais necessários para monitoramento pós-deploy

Deploy maduro depende dessas três disciplinas.

11. Mudanças de alto risco exigem mais validação

Alterações em:

- autenticação
- autorização
- persistência
- AI Platform
- tracking
- follow-up
- integrações externas
- infraestrutura operacional

devem ser promovidas com nível adicional de cuidado.

12. Deploy não deve ser canal de correção estrutural

Problemas de arquitetura, testes, segurança ou observabilidade não devem ser “aceitos por enquanto” e empurrados para produção.

Deploy existe para publicar mudanças maduras, não para validar esperança.

13. Estado pós-deploy deve ser verificável

Toda promoção relevante deve permitir checagem pós-deploy, incluindo quando aplicável:

- health do sistema
- comportamento das rotas críticas
- funcionamento do dashboard
- integridade da persistência
- sinais da AI Platform
- tracking e follow-up

Deploy sem verificação pós-publicação é incompleto.

14. Feature release e code release não são sempre a mesma coisa

Mesmo quando a plataforma ainda for simples, a engenharia deve reconhecer conceitualmente que:

- código pode ser publicado
- comportamento pode ser habilitado depois
- rollout pode ser progressivo

Isso ajuda a preparar o sistema para maior sofisticação futura.

15. A plataforma deve evoluir para suportar CI/CD sem depender dele hoje

Mesmo antes de pipeline robusto completo, as decisões atuais devem facilitar futura automação de:

- validação
- build
- test
- release
- rollback

Não bloquear o futuro por processos manuais frágeis.

Deployment Decision Framework

Antes de promover mudança relevante, deve ser possível responder:

1. o que exatamente está sendo publicado?
2. esse estado já foi validado em `develop`?
3. qual risco operacional essa mudança traz?
4. existem migrations ou efeitos irreversíveis?
5. quais segredos ou configurações de ambiente são afetados?
6. como o deploy será observado após a promoção?
7. qual plano de rollback existe?
8. existe alternativa de rollout mais segura?
9. essa promoção mantém coerência entre branches e ambientes?
10. vale a pena promover isso agora?

Deployment Anti-Patterns

Nunca:

- publicar direto em `main` como fluxo normal
- compartilhar segredos entre ambientes
- fazer deploy sem plano de rollback
- tratar migration como detalhe
- promover mudança sem observabilidade pós-release
- depender de passos manuais obscuros
- usar produção como ambiente inicial de validação
- publicar mudanças críticas sem testes compatíveis
- reconciliar branches de forma negligente
- aceitar estado não rastreável de release

Future Compatibility

A política de deploy deve permanecer preparada para:

- staging formal
- produção formal
- CI/CD
- múltiplos ambientes
- múltiplos runtimes
- workers

- filas
- banco remoto
- deploy blue/green ou progressivo
- feature flags

Preparação futura não significa implementar tudo agora.

Significa não criar um processo de release que colapse assim que a plataforma crescer.

Definition of Done

Uma mudança só pode ser considerada pronta para deploy quando, de forma proporcional ao seu escopo:

- foi integrada corretamente em `develop`
- está madura para promoção a `main`
- possui validação compatível com o risco
- respeita segurança, testes e observabilidade
- possui plano de rollback razoável
- trata migration com cuidado quando necessário
- possui rastreabilidade suficiente
- evita anti-patterns de release
- preserva compatibilidade futura do processo operacional

Checklist

- A mudança entrou primeiro em `develop`?
- Está clara a intenção da promoção?
- O risco operacional foi avaliado?
- Há segredos ou configuração de ambiente impactados?
- Há migration ou efeito irreversível?
- O pós-deploy será observável?
- Existe rollback viável?
- A promoção evita anti-patterns conhecidos?
- A coerência entre branches foi preservada?
- A entrega atende ao Definition of Done?

Observações

Deployment Engineering no GrowthBot DE deve ser lido como disciplina de segurança operacional.

Publicar software não é apenas mover código entre branches.

É promover comportamento novo em uma plataforma que precisa continuar confiável, auditável e evolutiva.

Code Review

Code Review

Este documento define a política oficial de Code Review do GrowthBot DE. Ele estabelece como alterações devem ser avaliadas antes de serem tratadas como mudanças aceitáveis na plataforma.

Objetivo

Definir as regras obrigatórias de revisão de código do GrowthBot DE.

O objetivo deste documento é garantir qualidade técnica, segurança de mudança, consistência arquitetural e disciplina operacional antes da promoção de alterações entre branches e ambientes.

Escopo

Este documento cobre:

- critérios de revisão
- alinhamento com o handbook
- análise de risco
- revisão de comportamento
- revisão de arquitetura
- revisão de segurança
- revisão de testes
- revisão de observabilidade
- disciplina entre `develop` e `main`

Regras

1. Code review é mecanismo de proteção estrutural

Review não existe apenas para aprovar estilo ou sintaxe.

Ele deve proteger:

- arquitetura
- produto
- segurança
- testabilidade
- observabilidade
- coerência operacional

2. Toda revisão deve considerar o handbook

Uma alteração não pode ser avaliada isoladamente do Engineering Handbook.

Revisão deve considerar alinhamento com:

- missão
- arquitetura
- product engineering
- ai platform
- security
- testing
- observability

- backend, frontend, performance e deployment quando aplicável

3. Review deve focar comportamento e risco primeiro

A ordem correta de revisão é:

- comportamento
- risco
- regressão
- arquitetura
- segurança
- observabilidade
- testes
- estilo e detalhes menores

Detalhes cosméticos não podem ocupar o lugar de riscos reais.

4. Branch flow continua obrigatório

O fluxo oficial do projeto continua sendo:

- desenvolvimento e integração em `develop`
- promoção controlada para `main`

Review deve reforçar esse fluxo, não enfraquecê-lo.

5. Review deve avaliar impacto, não apenas diff local

Uma boa revisão considera:

- o que mudou
- por que mudou
- o que isso pode quebrar
- qual parte do sistema foi afetada
- qual risco futuro foi criado

Sem análise de impacto, review fica superficial.

6. Mudanças críticas exigem revisão mais rigorosa

Alterações em áreas como:

- autenticação
- autorização
- persistência
- AI Platform
- tracking
- follow-up
- observabilidade
- deploy
- integrações externas

devem receber escrutínio adicional.

7. Code review deve exigir testes proporcionais ao risco

Nem toda mudança precisa da mesma profundidade de teste.

Mas mudanças relevantes não devem ser aceitas sem validação compatível com impacto.

8. Review deve proteger fronteiras arquiteturais

Se a alteração viola boundaries, mistura camadas, aumenta acoplamento ou esconde lógica estrutural em pontos errados, isso deve ser apontado como problema real.

9. Review deve considerar observabilidade e operabilidade

Alterações que afetam comportamento relevante devem ser avaliadas também sob a perspectiva de:

- logs
- métricas
- rastreabilidade
- diagnóstico
- impacto operacional

10. Aprovar código não significa aprovar pressa

Velocidade é importante.

Mas review não deve sacrificar critérios fundamentais em nome de conveniência imediata.

Code Review Decision Framework

Antes de aprovar mudança relevante, deve ser possível responder:

1. essa alteração resolve o problema certo?
2. ela respeita o handbook?
3. há risco de regressão?
4. a arquitetura ficou melhor, igual ou pior?
5. segurança foi preservada?
6. observabilidade e operação foram consideradas?
7. testes são proporcionais ao risco?
8. existe mudança sensível em persistência, IA ou integração?
9. a mudança está madura para `develop` ou para promoção posterior?
10. o que pode quebrar depois dessa alteração?

Code Review Anti-Patterns

Nunca:

- revisar apenas estilo e ignorar comportamento
- aprovar sem entender impacto
- ignorar quebra de boundary
- aceitar ausência de testes sem justificativa
- tratar risco de segurança como detalhe
- ignorar observabilidade em mudança relevante
- normalizar mudança direta em `main`
- reduzir review a checklist vazia
- confundir rapidez com qualidade
- aprovar por confiança pessoal em vez de evidência técnica

Future Compatibility

A política de review deve permanecer preparada para:

- equipe maior
- múltiplos revisores
- PRs mais complexos
- pipelines formais
- mudanças multi-runtime
- deploys mais sofisticados
- integração com CI/CD

Definition of Done

Uma alteração só pode ser considerada suficientemente revisada quando, de forma proporcional ao seu escopo:

- respeita o handbook
- tem comportamento compreendido
- teve risco avaliado
- preserva arquitetura
- preserva segurança
- possui testes compatíveis
- considera observabilidade e operação
- está coerente com o fluxo `develop -> main`
- evita anti-patterns de review

Checklist

- O problema e a solução estão claros?
- A mudança respeita o handbook?
- Há risco estrutural, funcional ou operacional?
- Security foi considerada?
- Testing foi considerado?
- Observability foi considerada?
- Há boundary quebrado ou acoplamento indevido?
- A mudança está correta para `develop`?
- A promoção futura para `main` permanece segura?
- A entrega atende ao Definition of Done?

Observações

Code Review no GrowthBot DE deve ser tratado como filtro de qualidade e coerência da plataforma.

A revisão correta não apenas encontra bugs.

Ela protege a direção do produto e a saúde da engenharia.

Engineering Glossary

Engineering Glossary

Este documento define o glossário oficial de termos de engenharia do GrowthBot DE. Ele existe para reduzir ambiguidade conceitual entre produto, arquitetura, IA, backend, segurança, testes e operação.

Objetivo

Definir linguagem comum para o Engineering Handbook.

Escopo

Este documento cobre termos recorrentes do projeto, seus significados oficiais e o uso esperado dentro da documentação e das decisões técnicas.

Regras

1. Termos críticos devem ter significado estável

Quando um termo relevante aparecer no handbook, ele deve ser entendido com o mesmo significado ao longo dos documentos.

2. O glossário reduz ambiguidade

Quando houver risco de interpretações múltiplas, este documento deve servir como referência conceitual.

3. O glossário deve evoluir com disciplina

Novos termos só devem entrar quando houver uso recorrente ou valor real de alinhamento.

4. Termos preferenciais do projeto

- **AI Sales Platform** : identidade oficial do GrowthBot DE
- **Product Engineering** : disciplina que conecta produto, engenharia, experiência e operação
- **AI Platform** : camada responsável por modelos, prompts, contexto, tools, MCPs e inferência
- **Observability** : capacidade de entender comportamento do sistema por sinais externos confiáveis
- **Security Engineering** : disciplina que protege dados, acessos, integrações e operação
- **Testing Engineering** : disciplina que protege comportamento crítico e reduz regressões
- **Deployment Engineering** : disciplina de promoção, rollout, rollback e gestão de ambientes
- **Agent Chain** : coordenação multi-etapas entre agentes com papéis explícitos
- **Use Case** : intenção de negócio executável pelo sistema
- **Repository** : fronteira de acesso a persistência
- **Handler** : ponto de orquestração entre entrada e caso de uso
- **Fallback** : comportamento alternativo seguro quando o fluxo principal falha
- **Idempotência** : capacidade de repetir uma operação sem produzir efeito indevido adicional
- **Correlation ID** : identificador para correlacionar eventos de um mesmo fluxo

Engineering Glossary Decision Framework

Antes de adicionar termo novo ao glossário, deve ser possível responder:

1. o termo é recorrente no handbook?
2. existe ambiguidade real sem ele?
3. o significado proposto melhora alinhamento entre documentos?
4. esse termo deve ser oficial ou apenas contextual?

Engineering Glossary Anti-Patterns

Nunca:

- adicionar termo desnecessário
- manter termo ambíguo sem definição
- usar nomes diferentes para o mesmo conceito oficial
- transformar o glossário em lista inflada sem utilidade real

Future Compatibility

O glossário deve permanecer preparado para acompanhar a evolução do handbook sem perder clareza, estabilidade conceitual e utilidade prática.

Definition of Done

Um termo só pode ser considerado maduro para entrar no glossário quando:

- é recorrente
- reduz ambiguidade real
- possui significado claro
- não duplica conceito já existente

Checklist

- O termo já existe com significado oficial?
- Há ambiguidade que este documento precisa resolver?
- O novo termo é recorrente o suficiente para entrar no glossário?
- O uso do termo está consistente com o handbook?

Observações

O glossário não substitui os documentos específicos.

Ele existe para alinhar linguagem.

Quando houver conflito entre uso informal de um termo e seu significado oficial no handbook, o significado oficial deve prevalecer.

Agent Chain

Agent Chain

Este documento define a política oficial de Agent Chain Engineering do GrowthBot DE. Ele estabelece como cadeias de agentes, coordenação entre agentes e decomposição multi-etapas devem ser tratadas na evolução futura da plataforma.

Objetivo

Definir as regras obrigatórias para introdução e evolução de agent chains no GrowthBot DE.

O objetivo deste documento é impedir que a plataforma evolua para múltiplos agentes sem governança, sem fronteiras claras e sem capacidade de observação e controle.

Escopo

Este documento cobre:

- papel das agent chains
- coordenação entre agentes
- responsabilidade individual de agentes
- handoff entre etapas
- uso de tools e MCPs
- limites operacionais
- segurança, observabilidade e testes
- fallback e contenção

Regras

1. Agent chain não é default

O GrowthBot DE deve preferir fluxo simples e determinístico quando isso resolver bem o problema.

Agent chain só deve existir quando houver necessidade real de coordenação multi-etapas.

2. Cada agente deve ter papel explícito

Nenhum agente deve existir como executor genérico de qualquer tarefa.

Cada agente precisa ter:

- objetivo claro
- entradas claras
- saídas claras
- ferramentas autorizadas
- limites operacionais

3. A cadeia deve ser compreensível

Deve ser possível explicar:

- quem inicia
- quem delega
- quem decide
- quem executa
- quem finaliza

Se a cadeia não é explicável, ela não está madura.

4. Handoffs devem ser controlados

Toda passagem entre agentes deve operar com contexto suficiente, mas minimizado.

Não transferir contexto irrestrito por conveniência.

5. Agent chain deve respeitar AI Platform, Security e Observability

Mais agentes significam mais risco, mais custo e mais opacidade.

Por isso, cadeias multiagente exigem maior rigor de segurança, rastreabilidade e controle.

6. Falha de um agente não pode colapsar tudo silenciosamente

A cadeia deve prever:

- fallback
- interrupção controlada
- devolução de erro compreensível
- rastreabilidade do ponto de falha

7. Agent chains devem ser testáveis

Mesmo que a coordenação seja sofisticada, o sistema deve manter capacidade de testar handoffs, decisões e falhas relevantes.

Agent Chain Decision Framework

Antes de introduzir ou alterar uma agent chain, deve ser possível responder:

1. por que um fluxo simples não basta?
2. qual papel cada agente exerce?
3. qual handoff existe entre eles?
4. quais tools e MCPs cada agente pode usar?
5. qual risco operacional isso adiciona?
6. como a cadeia será observada?
7. como falhas serão tratadas?
8. essa complexidade é justificável?

Agent Chain Anti-Patterns

Nunca:

- criar múltiplos agentes sem necessidade real
- usar agente genérico para tudo
- permitir handoff opaco
- dar ferramentas amplas demais a todos os agentes
- aceitar cadeia sem teste, sem logs e sem fallback
- usar multiagente apenas porque parece sofisticado

Future Compatibility

Este documento deve permanecer preparado para:

- agentes especializados
- planning agents
- coordenação com tools e MCPs
- workers
- filas
- runtimes múltiplos

Definition of Done

Uma alteração relevante em agent chain só pode ser considerada concluída quando, de forma proporcional ao seu escopo:

- possui justificativa clara
- define papéis e handoffs explicitamente
- limita tools e escopo de cada agente
- respeita Security, AI Platform, Testing e Observability
- possui fallback e rastreabilidade
- evita anti-patterns conhecidos

Checklist

- Há necessidade real de chain?
- Papéis estão claros?
- Handoffs estão claros?
- Tools e MCPs estão limitados?
- Existe fallback?
- Há observabilidade suficiente?
- A complexidade é proporcional?
- A entrega atende ao Definition of Done?

Observações

Agent Chain Engineering no GrowthBot DE deve ser tratado como evolução controlada, não como default de sofisticação.

Mais agentes não significam automaticamente melhor produto.